TÉCNICO LISBOA

# Quantum Computation for the Prediction of Cancer Genes Based on Network Data

Quantum Multi-Order Moment Embedding for Graph-Based Node Classification

**Patrícia Simões Marques**

Thesis to obtain the Master of Science Degree in

## Data Science and Engineering

Supervisors: Dr. Bruno Gabriel Coelho Coutinho
Dr. Andreas Miroslaus Wichert

### Examination Committee

Chairperson: Prof. Name of the Chairperson
Supervisor: Dr. Bruno Gabriel Coelho Coutinho
Members of the Committee: Prof. Name of First Committee Member
Dr. Name of Second Committee Member
Eng. Name of Third Committee Member

**November 2025**

This work was created using LaTeX typesetting language
in the Overleaf environment (www.overleaf.com).

# Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

# Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

I would also like to acknowledge my dissertation supervisors Prof. Some Name and Prof. Some Other Name for their insight, support and sharing of knowledge that has made this Thesis possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life. Thank you.

To each and every one of you – Thank you.

# Abstract

Quantum computing holds promise for learning from complex networks, where capturing informative local patterns, such as cancer driver genes in protein-protein interaction graphs, can be critical for high-impact inference tasks. However, existing quantum machine learning (QML) approaches often overlook the gate complexity of initial state preparation, limiting their feasibility as end-to-end quantum solutions. In this work, we present a fully quantum pipeline for node classification in sparse networks with feature-labeled nodes. Our method is broadly applicable to graph-structured datasets where local node statistics provide significant discriminatory information.

We propose a framework that introduces a Quantum Multi-order Moment Embedding (QMME) protocol that encodes higher-order neighborhood statistics into amplitude-encoded quantum states representing higher-order neighborhood statistics. Classification is performed using a non-parametric, distance-based quantum circuit recently introduced in QML literature, which discriminates embedding states via quantum interference. The entire process - from embedding to inference - is performed within the quantum domain, requiring no classical optimization or training, and relying only on minimal data access assumptions alongside limited classical post-processing. Importantly, our pipeline explicitly addresses the state preparation step, ensuring that the embedding can be prepared with logarithmic qubit resources and polynomial-depth quantum circuits.

We validate our method through numerical simulations on synthetic graphs, analyzing classification performance alongside quantum resource metrics such as circuit depth and query complexity. This work establishes a proof-of-principle for end-to-end quantum learning on graph-structured data and lays the

groundwork for the application of quantum models to complex network inference tasks.

# Keywords

Quantum machine learning, Graph representation learning, Moment-based embeddings, Protein–protein interaction networks, Node classification, Amplitude encoding, Quantum graph algorithms, End-to-end quantum learning, Quantum state preparation

# Resumo

Quantum technologies are rapidly advancing, offering more secure communications and faster computations. It has been demonstrated that quantum computation can predict missing links in protein-protein interaction networks faster than classical computers [1,2]. This project explores the application of quantum algorithms to predict novel cancer genes through analysis of protein-protein interaction network datasets.

We define and design a quantum implementation of the classical algorithm proposed by Anja C. Gumpinger et al. [3] for cancer gene detection using protein-protein interaction network datasets: Moment embedding (capturing local network structure properties) and Machine learning (classifying potential cancer genes). This quantum implementation focuses on moment embedding through quantum computation and quantum machine learning. The project evaluates both the accuracy and computational efficiency of the developed quantum algorithm through analysis of query and gate complexity, considering the actual cost of implementing it on a universal quantum computer.

In this work, we propose a quantum algorithm for the task of classifying cancer driver genes through quantum machine learning on features derived from moment embeddings of mutation scores. Quantum Machine Learning (QML) offers the potential to outperform classical algorithms in processing structured data. However, many QML methods overlook the gate complexity of initial state preparation, limiting their practicality as end-to-end quantum solutions. In this work, we present a fully quantum pipeline for node classification in sparse networks with feature-labeled nodes, motivated by applications such as identifying cancer driver genes in protein-protein interaction networks. Our method is broadly applicable to graph-structured datasets where local node statistics provide significant discriminatory information. We propose a framework that introduces a Quantum Multi-order Moment Embedding (QMME) protocol that encodes higher-order neighborhood statistics into amplitude-encoded quantum states representing higher-order neighborhood statistics. Classification is performed using a non-parametric, distance-based quantum circuit recently introduced in QML literature, which discriminates embedding states via quantum interference. The entire process - from embedding to inference - is performed within the quantum domain, requiring no classical optimization or training, and relying only on minimal data access assumptions alongside limited classical post-processing. Importantly, our pipeline explicitly addresses the state preparation step, ensuring that the embedding can be prepared with logarithmic qubit resources

PM
Gostei deste tipo de intro: https://journals.a e https://journals.a

PM
Table as in The Kuramoto model in complex networks or Fault-tolerant noise guessing decoding of quantum random codes

and polynomial-depth quantum circuits. We validate our method through numerical simulations on synthetic graphs, analyzing classification performance alongside quantum resource metrics such as circuit depth and query complexity. This work establishes a proof-of-principle for end-to-end quantum learning on graph-structured data and lays the groundwork for the application of quantum models to complex network inference tasks.

In this paper, inspired by the method of moment in statistical theory, we propose to model neighbor's feature distribution with multi-order moments. We design a novel QGNN (Quantum Graph Neural Network) model, which includes a Quantum Multi-order Moments Embedding (QMME) module and a Classifier module. MM-GNN first calculates the multi-order moments of the neighbors for each node as signatures, and then uses a distance-based Classifier to allow for prediction [4].

# Palavras Chave

Graph representation learning, Quantum computing, Protein-protein interaction networks, Feature distribution, Moments, Genetics: Gene regulatory networks, protein–protein interaction networks.

# Contents


**1 Introduction** **1**

1.1 Overview . . . 3

1.2 State of the Art . . . 4

1.3 Objectives . . . 4

1.4 Outline . . . 4

**2 Background** **5**

2.1 Cancer Genome . . . 7

2.2 Complex Networks . . . 8

2.2.1 Graph Theory . . . 8

2.2.2 Protein-Protein Interaction Networks . . . 9

2.3 Quantum Computing . . . 10

2.3.1 Qubits and Quantum Operations . . . 10

2.3.2 Gate-Based Quantum Computing . . . 11

2.3.3 Block-Encodings . . . 12

2.4 Hybrid Computing . . . 14

2.4.1 Implicit Approach (Kernel Estimator + Classical SVM) . . . 15

2.4.2 Explicit Approach (Variational/Parameterized Quantum Circuits) . . . 16

**3 Quantum Multi-order Moments Embedding Protocol** **19**

3.1 Introduction . . . 21

3.2 Architecture Design . . . 24

3.3 Complexity Analysis . . . 30

**4 Quantum Classification Protocol** **33**

4.1 Introduction . . . 35

4.2 Quantum Classification . . . 36

4.2.1 Analogy with Classical Binary Classification: Support Vector Machine (SVM) with a Custom Kernel / But this is a parameter-free model, linear classifier. . . . 38

4.3 Complexity Analysis . . . 38

**5 Simulation and Implementation** **41**
5.1 Graph Augmentation . . . . . 43
5.2 Synthetic Dataset/Toy example . . . . . 44
5.3 Real Network Data . . . . . 44
**6 Concluding Remarks** **45**
6.1 Conclusions . . . . . 47
6.2 Limitations and Future Work . . . . . 47
**Bibliography** **49**
**A Code of Project** **55**
**B Drafts and A Large Table** **57**
B.1 Classification Unitary . . . . . 57
B.1.1 HHL-inspired linear regression/classification (Amplitude-Encoding the Class) . . . 58

# List of Figures

2.1 Adaptation System Behavior Test . . . . . 11

3.1 Quantum Multi-order Moment Embedding (QMME) Circuit. . . . . 23

5.1 Complete User Interface . . . . . 44

# List of Tables

2.1 Comparison of relevant works and their connection to our quantum embedding method . 8
2.2 Comparison between today's and target Architectures of Telcos . . . . . 9
2.3 A nice Spreadsheet using package "spreadtab". Notice the calculations. . . . . 12

3.1 Quantum Circuit Registers for Quantum Multi-order Moments Embedding Protocol (QMMEP) 21

B.1 Example table . . . . . 59
B.2 Example of a very long table spreading in several pages . . . . . 59
B.3 Sample Table. . . . . 59

# List of Symbols

| | | |
|---|---|---|
| $D_0$ | Initial Diameter | $\mu m$ |
| $N$ | Number of Nodes | |
| $n$ | Number of Qubits for Node Encoding | |

# List of Algorithms

5.1 Quantum Multi-order Moments Embedding Protocol Strategy . . . . . 43

# Listings

5.1 Example of a MPD file. . . . . 44
6.1 A listing with a Tikz picture overlayed . . . . . 47
A.1 `Matlab Function` . . . . . 55
A.2 `function.m` . . . . . 55
A.3 PYTHON Code . . . . . 56

# Acronyms

| | |
|---|---|
| **IQP** | Instantaneous Quantum Polynomial Time |
| **LSP** | Linear System Problem |
| **MGF** | Moment-Generating Function |
| **PQC** | Parameterized Quantum Circuit |
| **QC** | Quantum Classification |
| **QCP** | Quantum Classification Protocol |
| **QMME** | Quantum Multi-order Moments Embedding |
| **QMMEP** | Quantum Multi-order Moments Embedding Protocol |

# Glossary

**formula**

A mathematical expression . . . . . 4

**LaTeX**

LaTeX It is a mark up language specially suited for scientific documents as it can correctly format documents with all the typographical rules . . . . . 4

**mathematics**

Mathematics is what mathematicians do . . . . . 4

# 1

# Introduction

## Contents


1.1 Overview . . . . . 3
1.2 State of the Art . . . . . 4
1.3 Objectives . . . . . 4
1.4 Outline . . . . . 4

Pellentesque vel. Integer adipiscing congue metus.

Quantum algorithms and quantum error correcting codes promise quantum advantages in certain computational tasks over existing classical methods. Nevertheless, demonstration of the quantum advantage in practical and industrial applications remains on the agenda [5]. A discipline for which quantum techniques are expected to be beneficial is machine learning [1–7] [5].

Rui Cruz: You should always start a Chapter with an introductory text

## 1.1 Overview

Rui Cruz: Examples of techniques, tools, and packages: preserve for reference, by moving the respective blocks of text to the last Chapter of this template (or to a Chapter file that you know you will not use), until you finish your document.

Quantum computing is getting real [6]. Node-level tasks are concerned with predicting the identity or role of each node within a graph.

PM: Achei imensamente caricata esta expressão.

Example of using package `todo` for notes of authors. In this case the author Johnny is calling the attention for something at the specific place in the text.

Johnny: pointing out to the place

In this other case, another co-author is making a note about the citation for missing some bibliographic record [7–9].

Pete: You should cite also Pellentesque:2014

Quisque facilisis erat a dui. “Cras gravida.”

This is an example of Tracking ChangesXangesJO (in this case a replacement) by different authors in the document. The Text can additionally be modified by addingPT new text or by deleting ~~wrong~~MN inadequate text. Author can manipulate changes introduced by each author~~, as adequate~~MN ~~introduced by other authors~~PT.

Rui Cruz: notice here how to enquote correctly

Patrícia Marques: Use MM-GNN and Prediciton as two main sources of inspiration.

This project involves using quantum computing to predict data in network nodes, focusing on protein interactions and cancer. The protocol has these main steps: Analyzing moments and how information spreads, Using quantum machine learning to calculate the likelihood of cancer. Key parts of the project: Learning quantum computing in depth, Using a quantum method called QSVD (Quantum Singular Value Decomposer), Figuring out how complex and costly the protocol is to run, Working with SSH for secure connections.

The goal is to use quantum computing to better understand how proteins interact with cancer and potentially predict cancer risks.

## 1.2 State of the Art

Patrícia Marques: Introdução (com estado da arte) até 10 páginas.

## 1.3 Objectives

RC use of ACRONYM defined in file "Aconyms.tex"

Praesent mauris Quantum Classification Protocol (QCP) volutpat 3G/4G.

• Master advanced quantum computation foundations; such as qubitization techniques, quantum singular value transformation method and quantum circuit design and optimization • Develop expertise in quantum machine learning, such as classification algorithms for biological datata • Analyze algorithmic efficiency; query complexity evaluation, gate complexity analysis and classical vs quantum performance benchmarking • Design and implement a quantum algorithm for cancer gene prediction: – develop quantum versions of all three phases – evaluate computational advantages and limitations – implement using [: performed simulations of the experiment using the IBM quantum information science kit] Qiskit or similar software for both simulation and testing – validate using real and synthetic biological datasets – Algoritmo para originar os embeddings em redes reais (GitHub) e para graph-augmentation. Quase guia.

RC use of SYMBOL defined in file "Glossary.tex"

The initial diameter ($D_0$) of the tube corresponds to a surface area ($N$) of $122\mu m^2$. The typical $N$.

H.264/Quantum Multi-order Moments Embedding (QMME) [10, 11] null.

PM: Use Figure 3.1 instead of 3.1

You can use in-paragraph lists with this construct for: (a) first case; (b) second case; and (c) third case, making the text organized and fluid.

Notice: mathematics uses Formulas, well rendered in documents produced with LaTeX.

RC example of use of Glossaries

Expected Contributions • A novel quantum algorithm for cancer gene prediction • Comprehensive analysis of quantum advantages in biological network analysis • Documentation of methodology and results suitable for academic publication

## 1.4 Outline

Thesis structure. This thesis is is organized as follows: Chapter 1. In Chapter 2 etc. In Chapter 3 etc. Chapter 3 and includes chapter 4. Chapter 6 dolor.

# 2

# Background

## Contents


2.1 Cancer Genome . . . . . 7
2.2 Complex Networks . . . . . 8
2.3 Quantum Computing . . . . . 10
2.4 Hybrid Computing . . . . . 14

PM: Descrição técnica (background, tools, já definidas pré-tese) 5-10 pp. André Roque tem 17 pp. There is Related Work as well: Trabalho feito em redes e computação quântica (Previous Work, não necessariamente Related com o meu Problema): Prediction of Cancer Drives Genes e Quantum Machine Learning.

PM: Ir escrevendo já sumo do meu trabalho e, quando preciso, add background ("isto era preciso saber").

## 2.1 Cancer Genome

PM: No background ou validação, consoante use a rede real ou não. Explicar o que é o cancer genome. Dar insight suficiente/mínimo para se perceber a relação entre genes, proteínas, cancro, células.

Genes act in concert within biological networks, particularly through the proteins they encode. Protein–protein interaction (PPI) networks, where nodes represent proteins and edges represent physical or functional interactions, offer a framework to study gene relationships at the functional level. By mapping genes onto these networks via their protein products, researchers can analyze the topological and functional proximity of candidate cancer genes to known drivers, helping to identify novel rare drivers missed by mutation frequency alone.

DNA (Deoxyribonucleic Acid):molecule that carries genetic information. DNA lives in the nucleus of most cells, tightly packed into chromosomes. There's also a little DNA in the mitochondria. Each gene is made up of a sequence of DNA letters. Cancer genome: complete set of DNA, including all the mutations, found in the cells of a specific cancer. Tumor exome: collection of all exons (the coding regions of genes) from the DNA of a tumor.

- NetSig: Network Significance - Cancer genomes: mutated DNA in cancer cells from many patients. - Network-based discovery: rather than studying genes in isolation, look at how genes interact with each other in biological networks (like human protein-protein interaction networks) - be concrete, is it InWeb?. Instead of asking, *"Is this one gene mutated a lot?"* NetSig asks, *"Are this gene's neighbors in the network showing signs of being involved in cancer?"* Detect important but rarely mutated genes based on their context.

Cancer driver genes are genes whose alterations (mutations, copy number changes, or fusions) give cells a growth or survival advantage, contributing directly to cancer development.

Gene's PPI network: The set of proteins that are known to physically or functionally interact with the protein encoded by that gene.

https://www.lagelab.org/resources/

"An important feature of NetSig is that it is explicitly designed to disregard any mutation information on the gene being tested so that the signal comes from the genes network alone. This ensures that NetSig P values are fully independent of those from existing gene-based statistical test such as MutSig, Oncodrive, GISTIC and RAE (in fact, the MutSig and NetSig P values of the same genes are only modestly correlated, Pearson correlation coefficient = 0.05, data not shown)."

https://www.lagelab.org/resources/

https://www.cancer.gov/ccg/research/genome-sequencing/tcga

https://www.nature.com/articles/nature12912.pdf

https://www.nature.com/articles/nature12634.pdf

**Table 2.1:** Comparison of relevant works and their connection to our quantum embedding method

| | Core Idea | Relevance |
|---|---|---|
| NetSig (Horn et al., 2018) | Identifies cancer driver genes using aggregated mutation signals from direct network neighbors. | Inspires the use of graph-local statistics to infer node relevance; emphasizes network structure for signal detection. |
| Moment Propagation (2021) | Propagates higher-order statistical moments of mutation scores through biological networks. | Motivates the use of rich local descriptors (moments) beyond mean aggregation; forms the statistical basis for QMME. |
| This work (2025) | Encodes multi-order neighborhood moments into quantum states for node classification via fully quantum inference. | Introduces a novel quantum approach to graph embedding and classification, extending the ideas above into a quantum paradigm. |

https://www.nature.com/articles/nature12912.pdf

Recent network-based methods like NetSig (Horn et al., 2018) detect cancer driver genes by assessing mutation patterns within a gene's local network neighborhood, without prior knowledge of gene function. Although effective, NetSig operates in an unsupervised manner, relying on known cancer genes mainly for validation after the fact. In contrast, the 2021 study Prediction of Cancer Driver Genes through Network-Based Moment Propagation introduced a supervised framework that leverages labeled cancer genes to propagate local statistical features—such as mean and variance of mutation scores—across protein interaction networks for improved gene classification.

Building on these insights, we present a fully quantum, supervised approach that captures multi-order local statistics in protein-protein interaction graphs. Our Quantum Multi-order Moment Embedding (QMME) encodes intricate neighborhood information directly into quantum states, enabling end-to-end classification without classical optimization loops. The quantum pipeline offers computational advantages by efficiently representing high-dimensional graph statistics with logarithmic qubit resources and polynomial-depth circuits, scaling more favorably than classical methods as graph size and complexity grow. This positions QMME as a promising step toward leveraging quantum computing to advance cancer gene discovery on complex biological networks.

## 2.2 Complex Networks

### 2.2.1 Graph Theory

A network is a collection of interacting elements, typically represented as a graph. The human interactome — a comprehensive network of human protein-protein interactions (PPI) within the human cell (be concrete: InWeb?) — serves as a valuable resource for discovering new cancer-related genes and potential drug combinations. As the complexity of biological networks continues to grow, developing innovative algorithms for efficient network analysis becomes increasingly important.

PM

Quantum computing close to maths/CS: use 1/2 teoremas/lemas principais

**Theorem 2.2.1.** *Lema/Teorema/definition Central de Quantum Computing Let* $G = (V, E)$ *be a graph*

*and ($U_1$, $T_1$), ($U_2$, $T_2$),..., ($U_k$, $T_k$), be matrix. Then among all forests containing $T = \cup^k j = iTj$, there is an optimal one block-encoding $(u, v)$.*

**Theorem 2.2.2.** *Block-Encodings Let $G = (V, E)$ be a graph and ($U_1$, $T_1$), ($U_2$, $T_2$),..., ($U_k$, $T_k$), be matrix. Then among all forests containing $T = \cup^k j = iTj$, there is an optimal one block-encoding $(u, v)$.*

One elegant and memory-efficient way of representing sparse matrices is as adjacency lists.

### 2.2.2 Protein-Protein Interaction Networks

We consider a PPI network with $N$ nodes or vertices, with $N = 2^n$ where $n$ is an integer. The adjacency matrix is $s$-sparse and is represented by $A$, with $s = 2^m$ and $m \ll n$. In this context, $s = k_{\text{max}}$, the maximum degree of the network.

There exists a feature value for each vertex $v \in V$, denoted by $x_v \in \mathbb{R}$ or in vector notation by $\vec{x} \in \mathbb{R}^N$. We assume this feature value is normalized such that $x_v \in [0, 1]$ for all vertices $v \in V$. In vector notation, this corresponds to $\vec{x} \in [0, 1]^N$.

The $k$-hop neighborhood of a vertex $v \in V$ is the set of all genes that can be reached from $v$ along at least $k$ edges, denoted by $\mathcal{N}_v^k$. For a vertex $v$, we denote with $\mathcal{X}_v^k = \{x_u | u \in \mathcal{N}_k^v\}$ the feature values of vertices in $\mathcal{N}_v^k$. We denote by $\mathcal{X}^k$ the collection of sets $\mathcal{X}_v^k$ for all $v \in V$. [12]. Table 2.2 is automatically compressed to fit text width. You can use `https://www.tablesgenerator.com` to produce these tables, and then copy the LaTeX code generated to paste in the document.

PM: Em edição a isto, meter as definições numa tabela. Be concrete: InWeb?

**Table 2.2:** Comparison between today's and target Architectures of Telcos

| Today | | Target | |
|---|---|---|---|
| Rigid | Each evolutionary requirement involves development of multiple components, interfaces, platforms,etc. | Flexible | It is possible to modify or add new functionalities rapidly. |
| Slow | Development of a new application takes months or years. | Fast | Development of a new application takes weeks instead of months or years. |
| Closed | Limited integration with external environments. | Open | It is simple to integrate internal, applications with external entities. |
| Complex | Heterogeneous technologies, obsolescence, lack,of standards, high redundancy. | Standardised | Use of homogeneous architectural models. |
| Expensive | High Capex (for new service development) and,high,Opex (to ensure running of IT). | Cost-Effective | Capex and Opex are optimised. |

PM: redes complexas antes de computação quântica → dificuldades computacionais nas redes e operações matriciais

## 2.3 Quantum Computing

Quantum computation is based on two quantum phenomena: quantum interference and quantum entanglement [13].

### 2.3.1 Qubits and Quantum Operations

In classical computing, information is encoded in bits, which represent binary states — either 0 or 1. Quantum computing, on the other hand, uses quantum bits, or qubits, as its fundamental unit of information. A qubit is also a two-level system, similar to a classical bit, but with a key difference: due to the principles of quantum mechanics, a qubit can exist in a superposition of these two states. This means that a qubit can simultaneously be in a combination of $|0\rangle$ and $|1\rangle$, with corresponding complex probability amplitudes, $\alpha$ and $\beta$, such that the state normalization condition $|\alpha|^2 + |\beta|^2 = 1$ holds true.

PM: Enjoyed the Quantum-based representation of states explanation here [14]. Great ref for everything [15, 16]

The state of a qubit is represented in Dirac or bra-ket notation as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \tag{2.1}$$

where $\alpha$ and $\beta$ are complex numbers. A powerful way to visualize this quantum state is by re-expressing it in spherical coordinates, leading to the Bloch sphere representation. In this framework, the qubit state is given by:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle, \tag{2.2}$$

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \tag{2.1}$$

$$|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \tag{2.2}$$

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$$H|0\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix} = |+\rangle$$

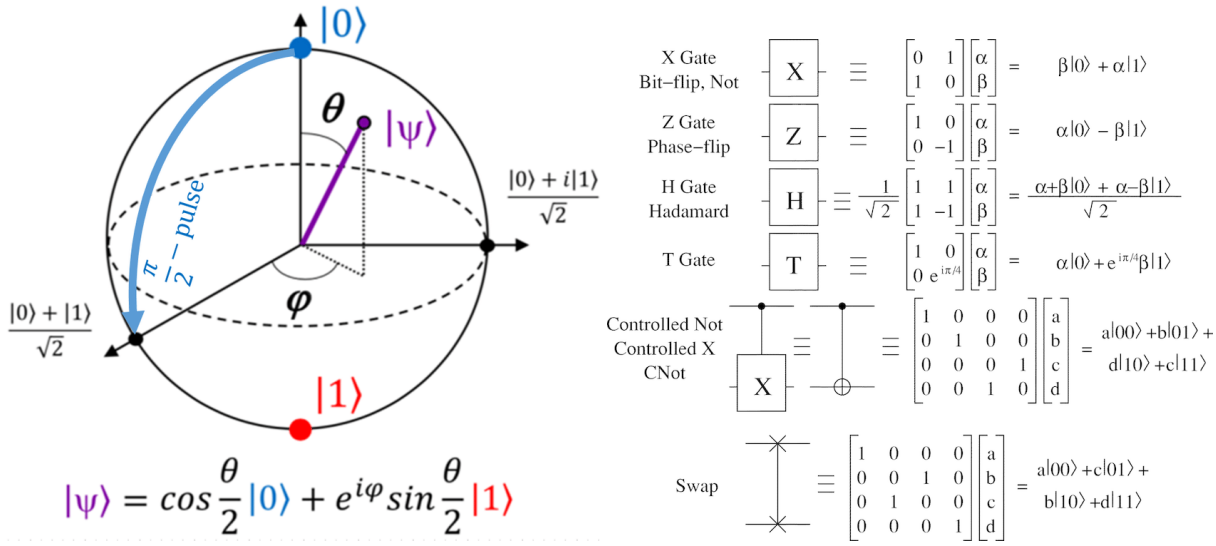
where $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi]$, providing a geometric representation of the qubit state on the Bloch sphere. Include $R_X, R_Y, R_Z$ (most important for me is the $R_Y$, which allows rotations around the Z axis,

and that is the circunference where we are acting)

This ability of qubits to exist in superposition allows quantum computers to perform certain computations exponentially faster than classical computers, particularly for problems involving parallelism and complex state manipulations.

PM: Complexity: Now, once we fix a universal set (any universal set) of quantum gates, we'll be interested in those circuits consisting of at most p(n) gates from our set, where p is a polynomial, and n is the number of bits of the problem instance we want to solve. We call these the polynomial-size quantum circuits. https://www.scottaaronson.com/democritus/lec10.html. Also, include Figure 7. Classical and quantum data-lookup oracles [60]. from 2024 QML review https://iopscience.iop.org/article/10.1088/1361-6633/ad7f69/pdf

Amplitude encoding as rigorous-ly described in [5] (Weighted nearest neighbors (parameter-free binary classification) arxiv.org).




**(a)** Bloch sphere: Adaptation System Test 4

**(b)** Qunatum gates: Adaptation System Test 5

**Figure 2.1:** Adaptation System Behavior Test

### 2.3.2 Gate-Based Quantum Computing

Every quantum operator ('gate') can be analytically expressed in the form of a matrix [14].

Summarize quantum computing in one sentence, it would be this: information is encoded in quantum states and processed using unitary operations. The challenge of quantum algorithms is to design and build these unitaries to perform interesting and useful tasks with the encoded information. My colleague Nathan Wiebe once told me that some of his early research was motivated by a simple question: Quantum computers can implement products of unitaries — after all, that's how we build circuits from a universal gate set. But what about sums of unitaries?

Quantum algorithms can be described by unitary transformations and projective measurements acting on the $2^n$-dimensional state vector of the register. In this context, unitary transformations are also called quantum gates [17].

Feature Rotation [18]. "Oracles" [19]. Query complexity **??**.

In [20], they enumerate detailed descriptions of the quantum computation model, and outline the basics using the terminology of quantum networks.

Suspendisse vestibulum dignissim quam [21]. Duis a eros. [22]. Table 2.3 illustrates the use of a Spreadsheet-like table producing calculations by columns and by lines (observe the code).

**Table 2.3:** A nice Spreadsheet using package "spreadtab". Notice the calculations.

| | | |
|---|---|---|
| 22 | 54 | 76 |
| 49 | 37 | 86 |
| 49 | 37 | 86 |
| 120 | 128 | 248 |

### 2.3.3 Block-Encodings

1. **Simulator-Friendly Encodings:**
   - Designed for efficient simulation on quantum computers.
   - Often use techniques like qubitization and Hamiltonian simulation.
2. **Block Encodings via Linear Combination of Unitaries (LCU):**
   - Represents a matrix as a weighted sum of unitary operations.
   - Requires ancilla qubits and controlled unitary operations.
   - Commonly used in Hamiltonian simulation and quantum algorithms.
3. **Block Encoding with Matrix Access Oracles:**
   - Useful for sparse and structured matrices.
   - Relies on oracle access to the matrix elements.
   - Can be implemented efficiently in graph-based quantum algorithms.

     PM: PPI networks.
4. **Sparse Matrix Block Encoding:**
   - Assumes the matrix has a limited number of nonzero entries per row.
   - Uses query-based access to efficiently construct the encoding.

- Efficient for quantum machine learning and differential equation solvers.

  PM: PPI networks.

5. **Tensor Product Block Encoding:**

   - Encodes a large matrix by decomposing it into tensor products of smaller matrices.
   - Often applied in quantum chemistry and quantum field theory.

PM: Block-encoding classical data into a quantum circuit has become a central component of modern quantum algorithms. (as in arXiv:2401.04234v1)
The task of constructing a block-encoding is nontrivial. In this paper we pose a different question; how can we efficiently transform application-relevant matrices into Walsh-Hadamard sparse matrices which compress well with FABLE? We begin by noting that generic sparse matrices are ubiquitous in quantum computing algorithms" "While the empirical contraction of quantum resources used by S-FABLE over FABLE is promising, both methods still require a large amount of classical pre-processing to compute the angles used in the rotation gates of the oracle" "For example, S-FABLE and LS-FABLE have difficulty block-encoding graph adjacency matrices with nonzero entries equal to one, and even more disappointingly unstructured nonnegative sparse matrices, requiring nearly all of their rotation gates to block-encode these classes of matrices to moderate accuracy.

Why making a quantum computer is extremely difficult? [14, 23] As inspecting the internal system state inevitably involves some physical interaction with the system itself, an important consequence of the indetermination principle is that any measurement operation on a quantum computer disrupts coherence [14].

Quantum machine learning (QML) approaches can be categorized into two main types: fully quantum computing-based methods and hybrid approaches that combine quantum computing components with classical neural networks. In general, a QML model which works with classical data requires three distinct blocks (Benedetti et al., 2019): data encoding, a variational circuit, and the measurement stage. The data encoding maps classical input data onto qubits, making it accessible to the quantum circuit. The variational circuit incorporates tuneable operations that adjust its parameters to fit the desired function. Finally, the measurement stage transforms the quantum state back into a real vector. Schuld and Petruccione, 2021 propose optimizing the measurement basis to learn functions, while Lloyd et al., 2020 suggests optimizing the variational circuit to maximize the distance between classes of points for classification tasks. Here, Lloyd et al. demonstrate the accuracy of their approach by replacing the fully connected layer of a convolutional neural network. Both approaches enable the realization of parameterized, optimizable functions on quantum circuits.

In the Fault-Tolerant Quantum Computation (FTQC) era, we envision a future where we have fully implemented quantum error correction (great QML 2024 review) [].

## 2.4 Hybrid Computing

**Before that: QGNNs and the difference from my work.** Use of Graphs: in my work, Graphs are implicit or used for feature extraction / embedding, in GNNs, Graphs are explicitly encoded in the model architecture (message-passing, graph convolutions, etc.). Quantum Feature Map: in my work, apply a fixed feature map to data (e.g., gene vectors), in GNNs, They define learnable quantum circuits for each node/message-passing operation [24] (QGNNs bible).

A link between the kernel method with feature maps and quantum computation was formally established by proposing to use quantum Hilbert spaces as feature spaces for data [25]. The ability of a quantum computer to efficiently access and manipulate data in the quantum feature space offers potential quantum speedups in machine learning [26] [This paragraph is from [27].] A quantum classifier is similar to a support vector machine that classifies data using learnable linear transformations in an exponentially large feature space.

One popular candidate application for quantum computing is machine learning [28]. Quantum machine learning may be defined as a field that investigates the use of quantum computing as a resource for machine learning [25]. In order for a quantum computer to have an advantage, one would like the state to be good at describing the solution of interest, while also difficult to prepare and/or sample from classically using currently known methods [29]. Near-term quantum devices require routines that are of shallow depth and robust against errors; the design paradigm of hybrid algorithms which integrate quantum and classical processing has therefore become increasingly important, cited from [30].

The central idea of hybrid algorithms is dividing the problem into two parts, each of which can be performed easily on a classical and a quantum computer [31]. A key point is, in the quantum part of the algorithm, using the quantum state as a feature space (quantum-enhanced feature space) [32]. This is called using quantum feature maps (for supervised learning) [25].

In [32], the algorithms solve a problem of supervised learning: the construction of a classifier. The first method is the the quantum variational classifier, uses a variational quantum circuit to classify the data, analogue to conventional SVMs, and corresponds to Section 2.4.2. The second method is a quantum kernel estimator, which estimates the kernel function on the quantum computer and optimizes a classical SVM, and corresponds to Section 2.4.1.

Note that quantum classifiers cannot provide a quantum advantage over a conventional SVM if the feature vector kernel can be computed efficiently on a classical computer. The authors of [32] highlight that a feature map that is hard to estimate classically is an important part of creating a quantum advantage, and conjecture that the additive error approximation of inner products generated from circuits with two Hadamard layers and diagonal gates is hard classically. That is why Instantaneous Quantum Polynomial Time (IQP) [33] embeddings are typically chosen. An example for a feature map that is believed to be classically intractable is an IQP, as [25] mentions.

In [25], they present mathematical formulations for the first strategy (Section 2.4.1) and to the second strategy (Section 2.4.2). The implicit approach uses the quantum device to evaluate the kernel function, while in the explicit approach, the model is solely computed by the quantum device.

PM: SWAP test relevance. We are following an explicit-like approach, ideally, the closed-form solution way.

### 2.4.1 Implicit Approach (Kernel Estimator + Classical SVM)

For the general theory of kernel-based quantum binary classifiers, [5] (very useful, including the abstract). Binary classification is an example of pattern recognition. The goal of this task can be described as, given a complex-valued labelled data set D = (x1, y1), . . . ,(xM, yM) ⊂ C N × 0, 1, finding the most likely class label of an unseen data point x˜ ∈ CN [5].

The implicit approach entails two steps: kernel estimation, via a quantum circuit, and a posterior classical classifier, as an SVM. Encode the training data $(y_v, |v\rangle)$ into a quantum feature space, using a quantum kernel function to classify the data [25, 34]. After this feature space mapper, use classical classifiers/techniques (like SVM) to perform the final classification [32]. Apply classical techniques to extract the final scores. Extract useful information by projecting

$$\sum_{v,c,i} U(v,c,i)\,|v\rangle\,|c\rangle\,|i\rangle \tag{2.3}$$

onto a computational basis. Apply controlled unitary transformations to weight different features, e.g., a weighted sum over the amplitudes corresponding to different $(c, i)$ combinations:

$$\sum_{v}\left(\sum_{c,i} f(c,i)U(v,c,i)\right)|v\rangle\,, \tag{2.4}$$

where $f(c, i)$ is a weighting function, like feature importance.

The kernel function measures the closeness of the test data to training data. With a nonlinear kernel, given a quantum test state, the protocol calculates the weighted power sum of the fidelities of quantum data in quantum parallel via a swap-test circuit followed by two single-qubit measurements.
Fidelity measures how close the transformed state is to the target state, and can be used as a transformation performance metric:

$$F(|y\rangle\,, U\,|\psi_{\text{final}}\rangle) = |\,\langle y|\,U\,|\psi_{\text{final}}\rangle\,|^2 \to 1 - P(\text{error}) \tag{2.5}$$

If fidelity is close to $1$, the transformation is accurate. If the fidelity is low, the operator $W$ should be adjusted to improve the transformation. This is typically done using quantum gradient descent, with a parameterized quantum circuit.

Quantum Amplitude Amplification, if the train labels $y_v$ are available, to align $|U\rangle$ with $|y\rangle$.

Classical Post-Processing: Process the measurements using classical methods, e.g., applying a matrix $M$ of size $N \times 8N/256N^3$ to derive the final scores for each node, e.g., post-aggregating the $8$ feature values into class predictions.

### 2.4.2 Explicit Approach (Variational/Parameterized Quantum Circuits)

PM
See Note's Chapter 9 for highlighted details

An interesting class of hybrid quantum-classical algorithms is called variational or parameterized quantum circuits. These can be trained with the help of a classical coprocessor [28]. Possibly the most well-known class of hybrid algorithms is that of variational circuits, which are parameter-dependent quantum circuits that can be optimized by a classical computer with regards to a given objective, cited from [30]. The quantum computer prepares the state, and a classical optimizer tunes the parameters.

In a training-based approach, i.e., quantum machine learning, $W$ or $U(\theta)$ is a trainable quantum operator, a learnable quantum circuit. The set of learnable parameters $\theta$ are trained using hybrid quantum-classical optimization, e.g., using quantum gradient descent.

PM
quantum gradient descent

The idea is to train the $W$ parameters to solve **??**. $W$ will use rotation gates, making this a Parameterized Quantum Circuit (PQC) learning problem. We optimize the weights through trial and error.

In [28] (circuit-centric quantum classifiers), they investigate a class of variational circuits for supervised machine learning, and propose a circuit architecture suitable for predicting class labels of quantumly encoded data via measurements of certain observables. The predicted class is retrieved by measuring a certain qubit in the state $U(\theta)\,|x\rangle$", with $x$ the input. These quantum classifiers use entanglement as a key resource.

In [31], there is a "new hybrid framework, called quantum circuit learning (QCL), for machine learning with a low-depth quantum circuit. "In QCL, we provide input data to a quantum circuit and iteratively tune the circuit parameters so that the optimized circuit gives the desired output." This QCL framework aims to perform supervised or unsupervised learning tasks.

In [35], something a bit different: the basis of training set labels, after an algorithm they explain, the expansion coefficients of the new state are the desired support vector machine parameters (in the training part). Quantum Support Vector Machines (QSVMs) (might) use quantum kernels for classification. In the test part, they construct a state that is a superposition of the $|\tilde{u}\rangle$ (basically the SVM parameters in the basis) and of the $|\tilde{x}\rangle$ (input data). They then perform a sawp test and measure with a success probability P. Then, if P is larger or smaller than $1/2$ determines whether the label is $+1$ or not.

In [30], Pennylane differenciates between classical and quantum nodes for the calculation of gradients. A classical node is a numerical computation, while a quantum node executes a variational circuit $U$ on a quantum device and returns an estimate of the expectation value of an observable $\hat{B}$, estimated by averaging $\hat{R}$ measurements.

PM: There are more ways to calculate gradients (e.g., finite differences). See Note's Chapter 9 for more.

## Variational Principle and Quantum Variational Optimization

The Variational Theorem/Principle states that, for any trial wavefunction $|\psi_{\text{trial}}\rangle$, the expectation value of the Hamiltonian $H$ is always greater than or equal to the ground state energy $E_0$:

$$\langle\psi_{\text{trial}}|H|\psi_{\text{trial}}\rangle \geq E_0. \tag{2.6}$$

This allows us to approximate the ground state energy $E_0$ by optimizing a parameterized trial state $|\psi_{\text{trial}}(\vec{q})\rangle$. We adjust parameters $\vec{q}$ to minimize the expectation value:

$$E_{\text{approx}} = \min_{\vec{q}} \langle\psi_{\text{trial}}(\vec{q})|H|\psi_{\text{trial}}(\vec{q})\rangle. \tag{2.7}$$

If we find the best choice of $\vec{q}$, the energy $E_{\text{approx}}$ is as close as possible to $E_0$. This is the foundation of Variational Quantum Eigensolvers (VQE) [29]:

1. Prepare a parameterized quantum state $|\psi(\vec{q})\rangle$.
2. Compute the expectation value $\langle H\rangle$.
3. Use classical optimization to adjust $\vec{q}$ and minimize $\langle H\rangle$.

# 3

# Quantum Multi-order Moments Embedding Protocol

## Contents


3.1 Introduction . . . . . 21

3.2 Architecture Design . . . . . 24

3.3 Complexity Analysis . . . . . 30

## 3.1 Introduction

From the perspective of quantum computing, a **quantum feature map** $x \mapsto |\phi(x)\rangle$ corresponds to a state preparation circuit $U_\phi(x)$ that acts on a ground or vacuum state $|0\cdots0\rangle$ of a Hilbert space $\mathcal{F}$ as $U_\phi(x)|0\cdots0\rangle = |\phi(x)\rangle$.

PM: I will call the quantum feature map, what happens after the superposition in $v$. Why? Because, for the state preparation of the QC, it's just the QMME + Feature Map on the test node (with the lil' note of $O_K$ and $O_L$ specific for that test node (no need to basis-encode the $v_{\text{test}}$.

We will call $U_\phi(x)$ the feature-embedding circuit [25]. The information encoding technique, and associated quantum feature map, we are using is amplitude encoding, as presented in the Supplement of [25]. Note the parameters $\phi$ depend ONLY on the network parameters (these come from the fact the oracles are specific for each network), and this does not change, if the network does not change.

**Amplitude encoding** is an approach to information encoding which is to associate normalized input vectors $x = (x_0, \ldots, x_{N-1})^T \in \mathbb{R}^N$ of dimension $N = 2^n$ with the amplitudes of an $n$-qubit state $|\psi_x\rangle$, such that $U_\phi : x \in \mathbb{R}^N \mapsto |\psi_x\rangle = \sum_{i=0}^{N-1} x_i|i\rangle$. As above, $|i\rangle$ denotes the $i$-th computational basis state. This choice corresponds to the linear kernel, $\kappa(x, x') = \langle\psi_x|\psi_{x'}\rangle = x^T x'$.

PM: Connect this to the idea of quantum kernels in Section 2.4.1.

PM: What I'm doing in this section is basically what they call applying a suitable "quantum" feature map $\Psi : D \to H$, where H is the Hilbert space of quantum states [28]. They also call it feature mapping in [32].

Consider using this **terminology for representation**: $\tilde{A_{ij}}$ is a d′-bit fixed point representation of $A_{ij}$. One can increase $d$ to obtain a higher precision with the cost of a more costly implementation [6].

PM: assume we have the binary version (and, for that reason, conversion should happen outside the gate, at maximum, it just cuts the precision). (1) If exact indexing is needed, use $\log_2$(different values)-bit integers and scale. (2) If binary fraction representation is needed, use at least $\log_2$(different values) fractional bits but accept small rounding errors.

The circuit uses the registers described in Table 3.1.

**Table 3.1:** Quantum Circuit Registers for Quantum Multi-order Moments Embedding Protocol (QMMEP)

| Register Index | Number of Qubits | Encoding Purpose | Gates Acting on Register |
|---|---|---|---|
| $i$ | 2 | Indexes Powers of Feature Values | Hadamard $H$ |
| $l$ | $\mathfrak{s}$ | Indexes First-Neighbor Nodes | Hadamard $H$, Oracle $O_L$ |
| $v$ | $n$ | Indexes Nodes $v$ | Hadamard $H$, Oracle $O_L$ |
| $k$ | $p_k \approx l$ | Represents $\tilde{k}_v$ | Oracle $O_K$ |
| $x$ | $p_x \approx l$ | Represents $\tilde{x}_v$ | Oracle $O_X$ |
| $a$ | 5 | Ancilla Qubits for Controlled Rotations | (Depends: Oracles or rotation Operators) |

Assuming $N$ features, we first consider simple normalizing state preparation:

$$(x_1, \ldots, x_N)^T \longrightarrow \bar{x} = \frac{1}{\chi}(x_1, \ldots, x_N, c_1, \ldots, c_P), \tag{3.1}$$

PM

Aqui ou noutro sítio provavelmente.

where $c_1, \ldots, c_P$ are some padding values that are constant across the data domain, and

$$\chi = \sqrt{\sum_j x_j^2 + \sum_k c_k^2}. \tag{3.2}$$

The padding dimension $P$ is selected such that the total dimensionality becomes a power of 2:

$$N + P = 2^n. \tag{3.3}$$

For a random variable $\mathcal{X}$, its $k$-th order **origin moment** is denoted as $\mathbb{E}(\mathcal{X}^k)$, where $\mathbb{E}(\mathcal{X}^k) = \int_{-\infty}^{\infty} x^k \cdot f(x; \theta_1, \ldots, \theta_k)\, dx$ [4]. The $k$-th term of the Taylor Expansion of the Moment-Generating Function (MGF) is the $k$-th order origin moment. An essential property is that MGF can uniquely determine the distribution [4].

We will use the notation $\mu$ to represent the arithmetic mean (or average) of a set of values, i.e., a known finite sample. While central and standardized moments (e.g., variance and skewness) are commonly used in statistical analysis, in this work we use origin moments, as they are more naturally compatible with our encoding scheme. In particular, origin moments are easier to represent in quantum amplitude encodings and avoid the need for explicit mean-centering or normalization operations, which are nontrivial in a quantum setting. Although origin moments do not isolate distributional shape in the same way as central moments, they still capture meaningful information about the data and are sufficient for the learning tasks considered here, we are assuming.

We now focus on the **amplitude encoding of** $\mu(\mathcal{X}_v^1)$. As such, we'll disregard register $i$ and consider only two ancilla qubits. Our objective is to achieve a final state $|\psi_{\text{final}}\rangle$ where the component of interest $\Psi_{\text{final}}$ (corresponding to when the ancilla qubits and register $\ell$ are all in the $|0\rangle$ state) satisfies

$$\langle v | \, |\Psi_{\text{final}}\rangle \propto \mu(\mathcal{X}_v^1) = \sum_{u \in \mathcal{N}_1^1} \frac{x_u}{|\mathcal{N}_1^1|}. \tag{3.4}$$

Let $\mu(\mathcal{X}_v^1)$ denote the mean of the feature values of the 1-hop neighborhood of node $v$. For all nodes $v \in V$, the vector of these means $\mu(\mathcal{X}^1) \in \mathbb{R}^N$ is given by:

$$\mu(\mathcal{X}^1) = K^{-1} \cdot A \cdot X \cdot \mathbf{1}, \tag{3.5}$$

where:

$X$ the diagonal matrix of node features, i.e., $X = \text{diag}(x_1, \ldots, x_N)$,

$\mathbf{1}$ the vector of ones in $\mathbb{R}^N$,

$A$ the adjacency matrix of the graph,

$K$ the degree matrix, defined as $K = \text{diag}(|\mathcal{N}_1^1|, \ldots, |\mathcal{N}_1^N|)$, with $|\mathcal{N}_1^v|$ being the number of 1-hop neighbors of node $v$.

The formulation Equation (3.5) captures the average feature value among each node's immediate neighbors.

The embedding captures the average feature value of the first-neighbor nodes for each node, that is, $\mu(\mathcal{X}_v^1) \in \mathbb{R}$ or, considering all nodes $v \in V$, $\mu(\mathcal{X}^1) \in \mathbb{R}^N$. In particular,

$$\mu(\mathcal{X}_v^1) = \frac{1}{|\mathcal{N}_1^v|} A\vec{x}. \tag{3.6}$$

Similarly, for any $k$,

$$\mu(\mathcal{X}_v^k) = \frac{1}{|\mathcal{N}_k^v|} A^k\vec{x}. \tag{3.7}$$

Expressing the variance $\sigma^2$, skewness $\gamma(\mathcal{X}_v^k)$, and kurtosis $\kappa(\mathcal{X}_{v:i}^k)$ in terms of raw moments, each one depends only on the previous plus an extra term $\mu((\mathcal{X}_v^k)^i))$ with $i = 2, 3, 4$, respectively,

$$\mu((\mathcal{X}_v^k)^2) = \frac{1}{|\mathcal{N}_k^v|} A^k\left(\vec{x} \cdot \vec{x}\right), \tag{3.8}$$

$$\mu((\mathcal{X}_v^k)^3) = \frac{1}{|\mathcal{N}_k^v|} A^k\left(\vec{x} \cdot \vec{x} \cdot \vec{x}\right), \tag{3.9}$$

$$\mu((\mathcal{X}_v^k)^4) = \frac{1}{|\mathcal{N}_k^v|} A^k\left(\vec{x} \cdot \vec{x} \cdot \vec{x} \cdot \vec{x}\right). \tag{3.10}$$

Amplitude Embedding represented in Figure 3.1.

PM

Explain circuit: schema with state evolution.

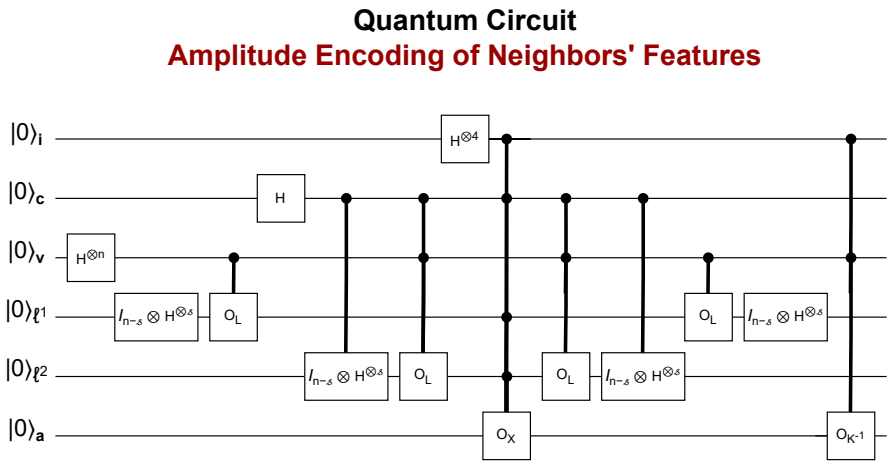



**Figure 3.1:** Quantum Multi-order Moment Embedding (QMME) Circuit.

PM

Descrições de figuras detalhadas, longas. e citá-las todas. Um leitor (possivelmente) vai olhar apenas para as figuras e descrições.

PM: Corrigir H em 2 qubits e não 4, em cima.

PM: Encode $X^{(i)}$ as $\prod_i U_X$ controlled on two extra ancilla qubits $|00\rangle_k$ (2 control qubits and 4 ancilla qubits). $\rightarrow$ I chose this one: Less oracles but more rotations (and qubits). Another option would be: encode $X^{(i)}$ as $U_{X^{(i)}}$ with $\theta = 2\arcsin(x_v^i)$. $\rightarrow$ More oracles (four and not one) but less registers needed (one and not four ancilla qubits).

## 3.2 Architecture Design

PM

Alternative titles: Quantum Amplitude Encoding / Amplitude Embedding of Neighborhood Moments

The framework we explore for efficient block-encoding of sparase and structured matrices is block encoding with matrix access oracles. We assume the existance of the oracles

PM: Write $U_A$ as factored as a product of simpler (Pauli, $R_y$) unitaries. Section 4 of [36]. "Possible to develop a general scheme to construct a block encoding and the corresponding quantum circuit for well-structured matrices. In particular, when the matrix A is sparse with a structured sparsity pattern and the nonzero matrix elements can also be well characterized, an efficient block encoding circuit for A can be constructed, as we will show in the next section." "Our goal is to construct a circuit that has a gate complexity of poly(n), i.e., a polynomial in n, [polylog N] at least for certain sparse and/or structured matrices with well defined sparsity patterns." "This assumption is of course somewhat restrictive" "it can generally be more difficult to explicitly construct quantum circuits for $O_C$ in these more general query models." [meter isto na complexidade]

PM

Approach as in 6.5. They also mention in Eq. 6.4 encode $\theta_i = \arccos(A_i)/\pi$. This step may require some additional work registers not shown here [37].

$$O_X |v\rangle |z\rangle = |v\rangle |z \oplus \tilde{x_v}\rangle . \tag{3.11}$$

$$O_L |\ell\rangle |v\rangle = |r(v,\ell)\rangle |v\rangle , \tag{3.12}$$

$$O_K |v\rangle |z\rangle = |v\rangle |z \oplus \tilde{k_v}\rangle . \tag{3.13}$$

PM: An oracle is the same (I think) as saying accessing qRAM. Important: I assume I have access to the second-degree oracle (or at least I should derive it). For $c = 1$, I could, instead of apply $O_{K^2}$: Apply $O_K$ to $q_u$ and $q_{k(u)}$. For the same $v$, sum the ($s$) values encoded in $q_{k(u)}$ and put the sum in $q_k(v)$. In the code, I chose to simply assume I have access to that oracle.

We assume that the feature vector entries are queried using an oracle using controlled rotations [36].

We assume the possibility to quickly recognize and locate the non-zero entries of the adjacency matrix of the graph, through access to the $(2n)$-qubit oracle with $v \in [N]$. Although $\ell \in [s]$, this index is expanded into an $n$-qubit state (e.g., let $\ell$ take the last $\mathfrak{s}$ qubits of the $n$-qubit register following the binary representation of integers) [37], Here, $r(v,\ell)$ denotes the $\ell$-th non-zero entry in the $v$-th row of $A$, or a flag if no such entry exists. The oracle can be interpreted as accessing the entries of the adjacency list, i.e., the $\ell$-th neighbor of the $v$-th node. A necessary condition for this query model is that $O_c$ is reversible, i.e., we can have $O_L^\dagger |r(v,l)\rangle |v\rangle = |l\rangle |v\rangle$ [37].

The following oracle is derived from the unitary oracles above. It basis-encodes the degree $k_v$ of vertex $v$ using $O(\log N) = O(n)$ applications of $O_L$ [1]. The vertex degree oracle $O_K$ can be constructed by initializing a counter register with $\lceil \log s \rceil$ qubits set to zero. For each vertex $v$, we repeatedly apply $O_L$ and increment the counter $\ell$ until a termination condition is reached. This implementation requires $O(|E|)$ applications of $O_L$.

Assume that each of the described queries, either classical or quantum, counts as $O(1)$ in the query complexity [2].

PM: Query calls and operations as in [2].

$$O_D |i\rangle |0_l\rangle = |i\rangle |D_{ii}\rangle , \quad i \in [N], \tag{3.14}$$

**Remark 3.2.1.** *We assume each diagonal entry of $D$ can be efficiently computed using a classical Boolean circuit of size $O(\text{polylog}(N))$ with $m = O(\text{polylog}(N))$ ancilla bits. In this case, we can construct a quantum circuit with $O(\text{polylog}(N) + l)$ gates and $O(\text{polylog}(N) + l)$ ancilla qubits to implement this oracle $O_D$ [38] (for simplicity assume the l-bit approximation of $D_{ii}$ is exact. which we MUST check).*

For simplicity, we omit these ancilla qubits in Section 3.2, since after reversing these quantum gates (uncomputing), their values will return to $|0_m\rangle$ at the end of the computation.

1. **Precompute directed edge list:** Define the directed edge set as:

$$E = \{(u, v) \mid A_{uv} = 1\}$$

   where $A$ is the adjacency matrix of the graph.

2. **For each directed edge** $(u \to v)$:

   (a) Query the oracle for $v$'s neighbors $w$.

   (b) For each $w$, query its neighbors $x$.

   (c) If $x \neq v$ and $x \neq u$, set:

$$B'_{(u\to v),(w\to x)} = 1$$

PM: Number of second-neighbors with backtracking or avoiding backtracking (depending on the sum of neighbors that is being considered). A nonbacktracking matrix is a specialized representation of a graph that encodes nonbacktracking walks, paths where no edge is traversed more than once in each direction. AGT: The non-backtracking matrix

The resulting matrix $B'$ allows step transitions through second neighbors while avoiding backtracking.

Describe amplitude encoding of the mean. We will also incorporate basis encoding to represent feature values in the computational basis when required. Each classical fractional feature value $x_v$ may

be represented with sufficient precision by encoding its binary representation through a transformation $x'_v = f_{\text{binary}}(x_v)$. We will consider $p_z$ qubits are required to represent $x'_v$ in the $z$-register.

Depois: Unitária de $A^2$ (aplicar $U_A$ outra vez como controlled gate num extra qubit)

**Rotation Operators: Implementation of the Controlled-Rotation of Ancilla Qubit [39]**

We start by considering the controlled rotation:

PM

See Chapter 7 of my notes: Oracle A.

$$W(a) = \begin{pmatrix} a & -\sqrt{1-a^2} \\ \sqrt{1-a^2} & a \end{pmatrix}. \tag{3.15}$$

This corresponds to a $y$-rotation by the angle $\theta = 2\arccos a$, i.e.,

$$W(a) = R_y(\theta), \quad \text{with } \theta = 2\arccos a. \tag{3.16}$$

The feature operator $F_X$ is a controlled-$W(x_v)$ and acts on a quantum state as

$$F_X |0\rangle |\tilde{x_v}\rangle = \left(x_v |0\rangle + \sqrt{1-|x_v|^2}\,|1\rangle\right) |\tilde{x_v}\rangle . \tag{3.17}$$

It is defined as:

$$F_X = \sum_{\tilde{x_v}} W(x_v) \otimes |\tilde{x_v}\rangle \langle \tilde{x_v}| . \tag{3.18}$$

**2.1 Fast inversion of diagonal, and general 1-sparse matrices [38]**

PM: Appendix C + Amplitude Amplification [19] [38]

The degree inversion operator $F_{K^{-1}}$ acts as

$$F_{K^{-1}} |0\rangle |\tilde{k_v}\rangle = \left(\frac{1}{k_v} |0\rangle + \sqrt{1 - \left|\frac{1}{k_v}\right|^2} |1\rangle\right) |\tilde{k_v}\rangle . \tag{3.19}$$

It is a controlled-$W\left(\frac{1}{k_v}\right)$, and its construction is given by:

$$F_{K^{-1}} = \sum_{\tilde{k_v}} W\left(\frac{1}{k_v}\right) \otimes |\tilde{k_v}\rangle \langle \tilde{k_v}| . \tag{3.20}$$

**Remark 3.2.2.** *We assume each diagonal entry of $D$ can be efficiently computed using a classical Boolean circuit of size $O(\text{polylog}(N))$ with $m = O(\text{polylog}(N))$ ancilla bits. In this case, we can construct a quantum circuit with $O(\text{polylog}(N) + l)$ gates and $O(\text{polylog}(N) + l)$ ancilla qubits to implement this oracle $O_D$ [38]).*

For simplicity, considering the input model and feature operators, we will focus on their essential actions. We'll disregard registers $k$ and $x$, and represent the operations as

$$\tilde{O_X} |0\rangle |j\rangle = \left( x_j |0\rangle + \sqrt{1 - |x_j|^2} |1\rangle \right) |j\rangle , \tag{3.21}$$

$$O_{K^{-1}} |0\rangle |v\rangle = \left( \tfrac{1}{k_v} |0\rangle + \sqrt{1 - \left|\tfrac{1}{k_v}\right|^2} |1\rangle \right) |v\rangle . \tag{3.22}$$

## Quantum Circuit for Amplitude Encoding $\mu(\mathcal{X}_v^1)$

The algorithm consists of three principal phases: initialization, with an equal superposition of all possible node states; sum of first-order neighbors' feature values, for each basis state; and normalization by the node's degree.

## State Preparation

Figure 3.1 shows the block-encoding $U$ for the scaled embedding matrix $\mu(\mathcal{X}_v^1)$.

*Proof.*

$$|v, 0^{\otimes}\rangle \xrightarrow{I_{n+3} \otimes H^{\otimes \mathfrak{s}} \otimes I_{\mathfrak{s}+5}} \frac{1}{\sqrt{s}} |v, 0^{\otimes 3}\rangle \sum_{\ell_1 \in [s]} |\ell_1, 0^{\otimes \mathfrak{s}+5}\rangle \tag{3.23}$$

$$\xrightarrow{O_L} \frac{1}{\sqrt{s}} |v, 0^{\otimes 3}\rangle \sum_{\ell_1 \in [s]} |r(v, \ell_1), 0^{\otimes \mathfrak{s}+5}\rangle \tag{3.24}$$

$$\xrightarrow{I_{n+3+\mathfrak{s}} C- \otimes H^{\otimes \mathfrak{s}} \otimes I_5} \frac{1}{\sqrt{s}} |v\rangle \sum_{\ell_1 \in [s]} \sum_{c \in [2]} |c, 0^{\otimes 2}\rangle |r(v, \ell_1), 0^{\otimes \mathfrak{s}+5}\rangle \tag{3.25}$$

$$\xrightarrow{C-O_L} \frac{1}{\sqrt{s}} |v\rangle \sum_{\ell_1 \in [s]} \sum_{c \in [2]} \left( |0_c, 0^{\otimes 2}\rangle |r(v, \ell_1), 0^{\otimes \mathfrak{s}}\rangle + |1_c, 0^{\otimes 2}\rangle \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} |r(v, \ell_1), \ell_2\rangle \right) |0^{\otimes 5}\rangle \tag{3.26}$$

$$\xrightarrow{I_{n+1} + H^{\otimes 2} \otimes I_{2\mathfrak{s}+5}} \frac{1}{\sqrt{s}} |v\rangle \sum_{\ell_1 \in [s]} \sum_{c \in [2]} \sum_{i \in [4]} \left( |0_c, i\rangle |r(v, \ell_1), 0^{\otimes \mathfrak{s}}\rangle + |1_c, i\rangle \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} |r(v, \ell_1), \ell_2\rangle \right) |0^{\otimes 5}\rangle \tag{3.27}$$

$$\xrightarrow{C-O_X} \frac{1}{\sqrt{s}} |v\rangle \sum_{\ell_1 \in [s]} \sum_{c \in [2]} \sum_{i \in [4]} \left( x^i_{r(v,\ell_1)} |0_c, i\rangle |r(v, \ell_1), 0^{\otimes \mathfrak{s}}\rangle + x^i_{r(v,\ell_1)} |1_c, i\rangle \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} |r(v, \ell_1), \ell_2\rangle \right) |0^{\otimes 5}\rangle \tag{3.28}$$

PM: Approach as in [37] 6.42

We apply the first four gate sets to the source state:

$$|0\rangle_{a_k}|0\rangle_{a_x}|0^n\rangle_{l^1}|0^n\rangle_v \xrightarrow{H^{\otimes n}} \xrightarrow{I_{n-\mathfrak{s}}\otimes H^{\otimes \mathfrak{s}}} \xrightarrow{O_L} \tag{3.29}$$

$$\frac{1}{\sqrt{Ns}}|0\rangle|0\rangle\sum_{v\in[N]}\sum_{\ell\in[s]}|r(v,\ell)=j\rangle|v\rangle \xrightarrow{O_X} \tag{3.30}$$

$$\frac{1}{\sqrt{Ns}}|0\rangle\sum_{v\in[N]}\sum_{\ell\in[s]}\left(x_j|0\rangle+\sqrt{1-|x_j|^2}|1\rangle\right)|j\rangle|v\rangle \tag{3.31}$$

where $j = r(v,\ell)$.

We then apply $O_{K^{-1}}$, $I_{n-\mathfrak{s}}\otimes H^{\otimes\mathfrak{s}}$, and $O_L$ to the target state:

$$|0\rangle_{a_k}|0\rangle_{a_x}|0^n\rangle_{l^1}|v\rangle \xrightarrow{O_{K^{-1}}} \xrightarrow{I_{n-\mathfrak{s}}\otimes H^{\otimes\mathfrak{s}}} \xrightarrow{O_L} \tag{3.32}$$

$$\frac{1}{\sqrt{s}}\sum_{\ell'\in[s]}\left(\frac{1}{k_v}|0\rangle+\sqrt{1-\left|\frac{1}{k_v}\right|^2}|1\rangle\right)|0\rangle|r(v,\ell')=i\rangle|v\rangle\,. \tag{3.33}$$

Then, the inner product, for the simplest case, gives

$$\langle 0|_{a_k}\langle 0|_{a_x}\langle 0^n|_{l^1}\langle v|U|0\rangle_{a_k}|0\rangle_{a_x}|0^n\rangle_{l^1}|0^n\rangle_v = \frac{1}{s\sqrt{N}}\frac{1}{k_v}\sum_{l,l'}x_j\delta_{r(v,l),r(v,l')} \tag{3.34}$$

$$= \frac{1}{s\sqrt{N}}\frac{1}{k_v}\sum_{l}x_{r(v,l)} \tag{3.35}$$

$$= \frac{1}{s\sqrt{N}}\sum_{u\in\mathcal{N}_1^v}\frac{x_u}{|\mathcal{N}_1^v|}. \tag{3.36}$$

Generalizing, with $v\in[N]$for different $c\in\{1,2\}$ and $i\in\{1,2,3,4\}$, the final state is always in the subspace where $l^1$ and $l^2$ are $0^n$. Any measurement in these registers will yield $0^n$ with probability $1$ and

$$\langle 0^5|_a\langle 0^{2n}|_l\langle v|\langle c|\langle i|U|0^5\rangle_a|0^{2n}\rangle_l|0^n\rangle_v|0\rangle_c|0^2\rangle_i = \sum_{u\in\mathcal{N}_c^v}\frac{x_u^i}{|\mathcal{N}_c^v|}\cdot\left(\frac{\delta_{c,1}}{2s\sqrt{2N}}+\frac{\delta_{c,2}}{2s^2\sqrt{2N}}\right) \tag{3.37}$$

$$= \mu((\mathcal{X}_v^c)^i)\cdot\left(\frac{\delta_{c,1}}{2s\sqrt{2N}}+\frac{\delta_{c,2}}{2s^2\sqrt{2N}}\right). \tag{3.38}$$

Without measurements, the final state resulting from the QMMEP is

$$U\left|\psi_{\text{initial}}\right\rangle = \left|0^5\right\rangle_a \left|0^{2n}\right\rangle_l \sum_{i,c,v} \mu((\mathcal{X}_v^c)^i)\left|v\right\rangle \cdot \left(\frac{\delta_{c,1}}{2s\sqrt{2N}} + \frac{\delta_{c,2}}{2s^2\sqrt{2N}}\right)\left|c\right\rangle\left|i\right\rangle + \sum_{a\neq 0^5}\left|a\right\rangle_a\left|\vec{\phi}_a\right\rangle \tag{3.39}$$

$$= \left|0^5\right\rangle\left|0^{2n}\right\rangle \sum_{i,c,v} \mu((\mathcal{X}_v^c)^i)\left|v\right\rangle \cdot \left(f_1\delta_{c,1} + f_2\delta_{c,2}\right)\left|c\right\rangle\left|i\right\rangle + \sum_{a\neq 0^5}\left|a\right\rangle\left|\vec{\phi}_a\right\rangle \tag{3.40}$$

$$= \left|0^{5+2n}\right\rangle \sum_{i,c,v} \mu((\mathcal{X}_v^c)^i)\left|v\right\rangle \cdot \left(f_1\delta_{c,1} + f_2\delta_{c,2}\right)\left|c\right\rangle\left|i\right\rangle + \sum_{a\neq 0^5}\left|a\right\rangle\left|\vec{\phi}_a\right\rangle, \tag{3.41}$$

where:

$f_1 = \frac{1}{2s\sqrt{2N}}$, $f_2 = \frac{1}{2s^2\sqrt{2N}}$ Scale factors for $c = 1$ and $c = 2$, respectively.
$\vec{\phi}_a$ Considered "junk", i.e., sub-states of no interest. Additional terms that arise from ancilla rotations.

$$f_c = \begin{cases} \dfrac{1}{2s\sqrt{2N}} & \text{if } c = 1 \\ \dfrac{1}{2s^2\sqrt{2N}} & \text{if } c = 2 \end{cases} \tag{3.42}$$

In this equality, we have used that $A$ is Hermitian, and there exists a unique $\ell$ such that $j = r(v, l)$, as well as a unique $\ell'$ such that $i = r(j, l')$ [37]. Also note that it each

Note that, since the "dummy" neighbors of $v$ have feature value 0, the average feature value of the first-neighbor nodes for each of the $N$ nodes of interest is, indeed, encoded in the sub-state amplitude. Call it "orthogonal garbage": error components, or better: the "leakage" terms caused by using rotations based on non-unit amplitudes. □

In Equation (3.43), observe that if the node register $|v\rangle$ is not measured, the conditional distribution over $c$ (hop index) and $i$ (power index) registers is uniform. That is, each hop $c \in \{1, 2\}$ and each moment $i \in \{1, 2, 3, 4\}$ appears with equal probability. Therefore, the variation in the amplitudes of the full state arises entirely from the node index $v$. For fixed values of $c$ and $i$, the amplitude of the state depends solely on the node $v$, through the features $\mu((\mathcal{X}_v^c)^i)$. The encoded data is thus node-specific, while $c$ and $i$ serve as structured axes across which the same statistical moment is applied uniformly to all nodes.

**Conclusion:** Without measurements, the final state resulting from the QMMEP is

$$U\left|\psi_{\text{initial}}\right\rangle = \left|0^{5+2n}\right\rangle \sum_{i,c,v} \mu((\mathcal{X}_v^c)^i)\left|v\right\rangle \cdot \left(f_1\delta_{c,1} + f_2\delta_{c,2}\right)\left|c\right\rangle\left|i\right\rangle + \sum_{a\neq 0^5}\left|a\right\rangle\left|\vec{\phi}_a\right\rangle, \tag{3.43}$$

where $f_1 = \frac{1}{2s\sqrt{2N}}$ and $f_2 = \frac{1}{2s^2\sqrt{2N}}$, and $(\vec{\phi}_a)$ refers to "junk", i.e., sub-states within the superposition that are of no computational interest.

$$U\left|\psi_{\text{initial}}\right\rangle = \left|0^{5+2n}\right\rangle \sum_{i,c,v} \mu((\mathcal{X}_v^c)^i)\left|v\right\rangle \cdot (f_1\delta_{c,1} + f_2\delta_{c,2})\left|c\right\rangle\left|i\right\rangle + \sum_{a\neq 0^5}\left|a\right\rangle\left|\vec{\phi}_a\right\rangle \tag{3.44}$$

$$= \left|0^{5+2n}\right\rangle \sum_{v,c,i} \mu((\mathcal{X}_v^c)^i) f_c \left|v\right\rangle\left|c\right\rangle\left|i\right\rangle + \sum_{a\neq 0^5}\left|a\right\rangle\left|\vec{\phi}_a\right\rangle \tag{3.45}$$

$$= \left|0^{5+2n}\right\rangle \sum_{v,j} u_{v,j}\left|v\right\rangle\left|j\right\rangle + \sum_{a\neq 0^5}\left|a\right\rangle\left|\vec{\phi}_a\right\rangle \tag{3.46}$$

$$= \left|0^{5+2n}\right\rangle \sum_{v=1}^{N}\left|v\right\rangle\left|\vec{u}_v\right\rangle + \sum_{a\neq 0^5}\left|a\right\rangle\left|\vec{\phi}_a\right\rangle, \tag{3.47}$$

where $\left|\vec{u}_v\right\rangle = \sum_{j=1}^{8} u_{v,j}\left|j\right\rangle$. This quantum state $U\left|\psi_{\text{initial}}\right\rangle = \left|\psi_{\text{QMME}}\right\rangle$ stores both useful information (structured data) and "junk" ($\vec{\phi}_a$). The useful information is stored in the relevant part of the quantum state (non-junk), which looks like

$$\sum_{v,c,i} \mu((\mathcal{X}_v^c)^i) f_c \left|v\right\rangle\left|c\right\rangle\left|i\right\rangle = \sum_{v,c,i} U(v,c,i)\left|v\right\rangle\left|c\right\rangle\left|i\right\rangle, \tag{3.48}$$

with $U(v,c,i)$ the input features for classification, encoded in the quantum state amplitudes. For each $v$, there are $8$ feature values for $(c \in \{1,2\}, i = \{1,2,3,4\})$. This is the quantum feature representation.

## 3.3 Complexity Analysis

PM: 1. Gate count (numbr of qubits). 2. Gate depth (longest sequence of gates that must be applied sequentially). 3. Number of Work Qubits (Ancilla Qubits: extra qubits to store intermediate results). 4. Query Complexity (Oracle Calls).

PM: Write $U_A$ as factored as a product of simpler (Pauli, $R_y$) unitaries. Section 4 of [36]. "Possible to develop a general scheme to construct a block encoding and the corresponding quantum circuit for well-structured matrices. In particular, when the matrix A is sparse with a structured sparsity pattern and the nonzero matrix elements can also be well characterized, an efficient block encoding circuit for A can be constructed, as we will show in the next section." "Our goal is to construct a circuit that has a gate complexity of poly(n), i.e., a polynomial in n, [polylog N] at least for certain sparse and/or structured matrices with well defined sparsity patterns." "This assumption is of course somewhat restrictive" "it can generally be more difficult to explicitly construct quantum circuits for $O_C$ in these more general query models." [meter isto na complexidade]

Rotação controlada em $v$ no ancilla $a_1$: complexidade nas portas $O(n)$ com overhead "escondido"

está o basis encoding de $\theta_v$ (most likely em $O(\text{polylog } n)$ oracle calls:

$$|x'\rangle |0^n\rangle |0\rangle \to |x'\rangle |\arccos x'\rangle |0\rangle \tag{3.49}$$

PM

Single Equation: Use equation. Multiple Equations or Aligning: Use align. Centering Without Alignment: Use gather. Terminar com ponto final (enquadrado nas frases)

**Complexity Analysis of Oracle$_X$**

The oracle consists of two main operations:

1. $O_X$ **- Loading Feature Values**

$$O_X |v\rangle |0\rangle = |v\rangle |\tilde{x_v}\rangle \tag{3.50}$$

This operation maps a node index $v$ to its feature value $x_v$. The cost depends on how $x_v$ is stored:

- If stored classically and queried efficiently: $O(\text{polylog } N)$.
- If loaded using qRAM: $O(\log N)$ [40].

2. $F_X$ **- Controlled** $R_y$ **Rotation**

$$F_X |0\rangle |\tilde{x_v}\rangle = \left(x_v |0\rangle + \sqrt{1 - |x_v|^2} |1\rangle\right) |\tilde{x_v}\rangle \tag{3.51}$$

$C - R_y(\theta)$ gate with $\theta = 2 \arccos x_v$. Compute $\theta$ from $x_v$, then, apply a controlled rotation.

**Gate count:** Remember that

- $O_X$: If $x_v$ is loaded from a classical lookup table, this can be done in $O(\text{polylog } N)$ elementary gates.
- $F_X$: Requires computing $\theta$ and performing a controlled $R_y(\theta)$, which can be decomposed into [38]:
  - Computing $\theta = 2 \arccos x_v$: Quantum arithmetic circuit of $O(\text{polylog } N + l)$ elementary gates. Computed on-the-fly instead of stored. For fast inversion, Proposition 6 [38].
  - Controlled $R_y(\theta)$: Can be implemented using **1 CNOT and 2 single-qubit rotations**.

Thus, the **total gate count** is:

$$O(\text{polylog } N + l). \tag{3.52}$$

**Circuit depth:**

- $O_X$ and $F_X$ are applied sequentially.
- Depth is at most $O(\text{polylog } N + l)$.

**Work qubits:**

- $O_X$ requires $l$ qubits to store $x_v$.
- $F_X$ requires an ancilla qubit for the rotation.

Thus, the **total work qubits** required is:

$$O(l). \tag{3.53}$$

| Metric | Complexity Estimate |
|---|---|
| **Gate count** | $O(\text{polylog } N + l)$ |
| **Circuit depth** | $O(\text{polylog } N + l)$ |
| **Work qubits** | $O(l)$ |

Oracles that access entries of a diagonal matrix, or entries of a one-dimensional vector with $N$ entries, can be implemented with $O(\text{polylog}(N) + l)$ quantum gates and $O(\text{polylog}(N) + l)$ ancilla qubits Remark 3.2.1. For both $O_K$ and $O_x$, this implies a total of $O(\text{polylog}(N) + p_x + p_k)$ quantum gates and $O(\text{polylog}(N) + p_x + p_k)$ ancilla qubits to implement these oracles, with $p_x = \text{precision}(\tilde{x})$ and $p_k = \text{precision}(\tilde{k})$ Remark 3.2.1 (for simplicity assume the l-bit approximation of $D_{ii}$ is exact. which we MUST check).

Consider having precomputing basis encoding: feature values (or transformed values $\arccos(x_v)$), i.e., classically computed and stored in a structured quantum state.

Counting the complexity calculations twice, for $O_X$ and $O_K$; a number of $n + 4\mathfrak{s} + 2 + 1$ Hadamard gates adds $O(n + \mathfrak{s})$ into the gate count. Since $m \ll n$ and $n = \log N$. About the **number of work qubits**, we need $2 + 1 + n + 2\mathfrak{s} + 4$, without counting basis-encoding of $\arccos$ values, since we consider those are computed on-the-fly (and then included in gate complexity), and without counting $l$ of the basis encoding of feature and degree values. Thus, the **total work qubits for the QMME circuit** required is:

| Metric | Complexity Estimate |
|---|---|
| **Gate count** | $O(\text{polylog } N + l) = O(\text{poly } n + l)$ |
| **Circuit depth** | $O(\text{polylog } N + l = O(\text{poly } n + l))$ |
| **Work qubits** | $O(l + \mathfrak{s})$ |
| **Total number of qubits** | $O(n + l + \mathfrak{s}) = O(n + l)$ |

We can do complexity analysis, considering accessing input data $O(1)$, either just the oracles or the whole QMMEP.

PM: The $\mathfrak{s}$ scales with $N$, so include it. Add the possibility of the net not being sparse.

Our goal is to construct a circuit that has a gate complexity of poly(n), i.e., a polynomial in n, with $N = 2^n$ at least for certain sparse and/or structured matrices with well defined sparsity patterns [36]. On query calls, **it depends on how "complex"/component-based we define them, e.g.:** $O_X$ **including** $F_X$, or not. But, if it does not, we call each, and its inverse, once, and we query $F_X$ four times, and $F_{K^{-1}}$ once.

# 4

# Quantum Classification Protocol

## Contents


4.1 Introduction . . . . . 35

4.2 Quantum Classification . . . . . 36

4.3 Complexity Analysis . . . . . 38

## 4.1 Introduction

Suppose to deal with an unknown quantum state drawn from an ensemble where each state is labeled with respect to the class they came from. How well can we predict the class of this unknown quantum state? This problem is generally known as the quantum state discrimination problem or quantum classification problem when referred to machine learning [27, 41].

In the source paper [3], the authors exclusively utilized four classification algorithms to predict binary class labels: logistic regression, random forests, support vector machines (SVMs), and gradient boosting. Logistic regression models the posterior probability of a class by applying a sigmoid function to a linear combination of input features. Linear support vector machines determine a separating hyperplane that maximizes the margin between the two classes. Both logistic regression and linear SVMs involve the optimization of a weight vector and a bias term, and are therefore classified as linear models.

In this report, we explore the possibility of performing the entire classification process using quantum methods. **We investigate how far one can go under the assumption that the input data is available in a specific quantum-encoded format and that local node statistics provide significant discriminatory information.** How far can we go? Let us find out with a closed-form (deterministic) solution, without training or iterative models.

The state generated by the QMMEP is given by

$$U\left|\psi_{\text{initial}}\right\rangle = \left|0^{5+2n}\right\rangle \sum_{v=1}^{N} \left|v\right\rangle \left|\vec{u}_v\right\rangle + \sum_{a\neq 0^5} \left|a\right\rangle \left|\vec{\phi}_a\right\rangle, \tag{4.1}$$

$$\left|\psi_{\text{QMME}}\right\rangle = \sum_{v=1}^{N}\sum_{j=1}^{8} u_{v,j}\left|v\right\rangle\left|j\right\rangle = \sum_{v=1}^{N}\left|v\right\rangle\left|\vec{u}_v\right\rangle, \tag{4.2}$$

which is the input for the QCP, where:

$\left|\vec{u}_v\right\rangle$ Feature vector for the $v$-th data point, with $\left|\vec{u}_v\right\rangle = \sum_{j=1}^{8} u_{v,j}\left|j\right\rangle$,

$\left|v\right\rangle$ Index register for the $v$-th data point ($N$ in total),

$\left|j\right\rangle$ Feature register for the $j$-th feature ($8$ in total),

Note that I'm using 8 feature vectors but could use any other $M$ in theory.

$u_{v,j} \in \mathbb{R}$ Amplitude encoding the feature values,

**Normalization conditions:** $\sum_{v=1}^{N}\sum_{j=1}^{8} |u_{v,j}|^2 = 1$, $\sum_{v=1}^{N} |u_{v,j}|^2 = p(j)$, $\sum_{j=1}^{8} p(j) = 1$.

Each data point has a label $y_v^{\text{class}} \in \{0,1\}$, or a label $y_v \in [0,1]$. Assume the oracle for the training data

$$O_Y\left|v\right\rangle\left|z\right\rangle = \left|v\right\rangle\left|z \oplus y_v^{\text{class}}\right\rangle \tag{4.3}$$

is given. For all $v \in D_L$, where $D_L$ is the labeled set, the labels are already known, i.e., $O_Y : |v\rangle |0\rangle \mapsto |v\rangle |y_v^{\text{class}}\rangle$.

PM: The QMME resulting state is entangled in the sense that it cannot the factorized into products. The quantum state of each particle in a group cannot be described independently of the state of the others. I applied controlled rotations to one ancilla, which entangles that ancilla with the rest (data qubits), not necessarily the data qubits with each other. // The label qubits can be prepared with an $X^{y_m}$ gate applied to $|0\rangle$ [27] (!). In general a similarity measure, as given above, is a real-valued function D. In a quantum setting, the common similarity measures are the quantum state fidelity or the state overlap, i.e., the inner product. // Mention GNNs somewhere (maybe analogue? My activation function is like the $w_m$.

## 4.2 Quantum Classification

Hadarmard Classifier [42]:

$$|\psi^{\text{h}}\rangle = \frac{1}{\sqrt{2}} \sum_v \frac{1}{\sqrt{N}} \left( |0\rangle |\vec{u}_v\rangle + |1\rangle |\vec{\tilde{u}}\rangle \right) |y_v\rangle |v\rangle . \tag{4.4}$$

We don't simply apply the $O_L$ oracle as in Figure 3.1. Instead, on the new ancilla/control qubit $q_c$, apply an Hadamard. Then, apply a controlled version of $O_L$ providing that the control qubit $q_c$ is in state $|0\rangle$, such that $O_L : O_L |l\rangle |v\rangle |0\rangle = |r(v,l)\rangle |v\rangle |0\rangle$. Apply a controlled $\tilde{O}_L$ on $q_c$ being $1$, which gives the neighbors of $v^{\text{unseen}}$ $\tilde{O}_L : \tilde{O}_L |l\rangle |1\rangle = |r(v^{\text{unseen}}, l)\rangle |1\rangle$.

Swap-Test Classifier [27]:

$$|\psi^{\text{h}}\rangle = \sum_v \frac{1}{\sqrt{N}} |0\rangle |\vec{u}_v\rangle^{\otimes n} |\vec{\tilde{u}}\rangle^{\otimes n} |y_v\rangle |v\rangle \text{ with } n \in \mathbb{N} \text{ (polynomial kernel)} \tag{4.5}$$

$$= \sum_v \frac{1}{\sqrt{N}} |0\rangle |\vec{u}_v\rangle |\vec{\tilde{u}}\rangle |y_v\rangle |v\rangle \text{ for } n = 1 \text{ (simplest kernel).} \tag{4.6}$$

Add five extra ancilla qubits for the controlled rotations to block-encode $\vec{\tilde{u}}$. Use the $2n$ qubits (or more) to similarly apply the embedding scheme to achieve $|\vec{\tilde{u}}\rangle$.

PM: Building a kernel or inner product: measure or interfere only the —0⟩ branch.

Consider a binary classification function

$$f : \mathbb{R}^d \to \{0, 1\}, \tag{4.7}$$

which maps an input feature vector $\mathbf{x}^{(v)} \in \mathbb{R}^d$ to a predicted label $\hat{y}^{(v)} \in \{0, 1\}$. The prediction rule is structurally similar to classical kernel-based classifiers such as the Support Vector Machine (SVM).

Given a labeled training set $\{(\mathbf{x}_m, y_m)\}_{m=1}^M$ with $y_m \in \{0, 1\}$ (or $\{-1, +1\}$ under margin-based con-

ventions), and learned or fixed weights $w_m \in \mathbb{R}$, the quantum classifier defines a kernel via a quantum feature map:

$$|\tilde{\mathbf{x}}\rangle = U_\phi(\mathbf{x})|0\rangle,$$

which embeds classical inputs into a quantum Hilbert space via the unitary map $U_\phi$.

The classification is then determined through the expectation value of a two-qubit observable:

$$\langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle = \sum_{m=1}^{M} w_m \, y_m \, \Re \left( \langle \tilde{\mathbf{x}} | \tilde{\mathbf{x}}_m \rangle \right), \tag{4.8}$$

where $\sigma_z^{(a)}$ and $\sigma_z^{(l)}$ are Pauli-Z operators acting on the ancilla and label qubits, respectively. The inner product $\langle \tilde{\mathbf{x}} | \tilde{\mathbf{x}}_m \rangle$ is evaluated using a Hadamard test, giving access to its real component.

The final predicted label is obtained via:

$$\hat{y} = \frac{1}{2} \left( 1 - \operatorname{sgn} \left( \langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle \right) \right), \tag{4.9}$$

which outputs 0 when the expectation value is positive and 1 when negative. Like group-distance [43].

The SWAP-Test classifier [44] is a quantum kernel method. The test datum is assigned the label whose label vector achieves the highest overlap. There is here a similarity with k-nearest neighbors (kNN).

The paper [45] on multi-class Quantum Kernel-Based Classifier (posterior) also formulates well how an unitary encodes the test and training data into quantum states.

The article [46] did not ”invent” the SWAP test, but helped making it a standard tool in quantum information theory. The SWAP test classifier is directly based on that, i.e., estimating the fidelity (i.e., the inner product) between two quantum states.

The Hadamard test is a method capable of evaluating such a quantity on a quantum computer using just one additional qubit. This involves applying a control qubit with control over the extra qubit and applying the target unitary operator U to the quantum state. The idea is to apply a controlled U operation, with the control on that qubit, and target U on the quantum state. Then, one can measure from this single qubit state both the real and imaginary parts of the overlap [47, 48].

For a realistic hardware application of a controlled-SWAP gate, see [49], where the authors utilizes a hardware efficient circuit quantum acoustodynamic system to implement these gates (copied this from [48]).

### 4.2.1 Analogy with Classical Binary Classification: Support Vector Machine (SVM) with a Custom Kernel / But this is a parameter-free model, linear classifier.

The decision boundary in this quantum classifier is analogous to that of a classical kernel SVM, which defines a classifier of the form:

$$f(\mathbf{x}) = \sum_{m=1}^{M} \alpha_m \, y_m \, K(\mathbf{x}, \mathbf{x}_m) + b, \tag{4.10}$$

where $K(\cdot, \cdot)$ is a positive semidefinite kernel function, $\alpha_m$ are dual weights learned during training, and $b$ is a bias term. The predicted label is:

$$\hat{y} = \text{sign}(f(\mathbf{x})). \tag{4.11}$$

In this analogy, the quantum kernel

$$K_Q(\mathbf{x}, \mathbf{x}_m) := \Re\left(\langle \tilde{\mathbf{x}} | \tilde{\mathbf{x}}_m \rangle\right)$$

plays the role of the classical kernel $K$, and the Pauli-Z observable expectation captures the aggregated similarity measure used for classification. The Hadamard classifier thus functions similarly to a classical linear classifier in a high-dimensional feature space, where the kernel is defined by quantum state overlaps rather than explicit analytical functions.

## 4.3 Complexity Analysis

Both Figures 2.1(a) and 2.1(b)"perfect" tincidunt.

PM: I absolutely loved the writing on this review, and the clarity of the content. Get inspired by it "Likewise, the 'output problem' is faced when reading out data after being processed on a quantum device. Like the input problem, the output problem often causes a noticeable operational slowdown. (...) With the notable exceptions of least-squares fitting and quantum support vector machines, linear-algebra-based algorithms can also suffer from the output problem because desirable classical quantities such as the solution vector for HHL or the principal components for PCA are exponentially hard to estimate. (...) Ongoing work is needed to optimize such algorithms, provide better cost estimates and ultimately to understand the sort of quantum computer that we would need to provide useful quantum alternatives to classical machine learning." [40]

Whether we have a quantum speedup or not depends on a mathematical proof, if we take the computer science perspective, or on statistical evidence of a scaling advantage, if we take the finite size devices perspetive, more low-level or physics view. The latter perspective relies on the existence of a quantum computer - it's a benchmarking problem. Quantum speedups in machine learning (and general) are currently characterized using idealized measures from complexity theory: query complexity and

gate complexity [40]. [40] also argues that even very simple circuits are hard to simulate classically.

# 5

# Simulation and Implementation

## Contents


5.1 Graph Augmentation . . . . . 43

5.2 Synthetic Dataset/Toy example . . . . . 44

5.3 Real Network Data . . . . . 44

The device can either refer to quantum hardware or a classical simulator [30]. They also say ”quantum device (hardware or simulator)”.

We represent each node in the network with its $-\log_{10}$ transformed MutSig P-value [3, 50].

Binary string mapping - Big-Endian (normal) - Mathematica: Left-to-right, most significant bit first - Little-Endian (abnormal) - Python: Right-to-left, least significant bit first

## 5.1 Graph Augmentation

The use of a flag for cases where $\ell$ exceeds the number of non-zero entries ensures the oracle $O_L$ can handle all possible inputs consistently. For implementation purposes, one must define such a scheme. This scheme can be directly related to the problem of finding the minimal $r$-regular graph containing an induced subgraph of interest, [51] which, in our case, represents the PPI network.

We introduce $M$ ”dummy” nodes, with feature values set to $0$, to create an $r$-regular graph with $N+M$ nodes. The expansion ensures every node, including previously isolated ones, has exactly $s$ different neighbors (from the original PPI net or the dummy nodes). $O_L$ maintains unitarity in this enlarged network. The termination condition for $O_K$ remains valid, returning $k_v$ for the original subgraph.

Upper and lower boundary on $m$ Algorithm 5.1. asterisco → graph augmentation (coisa técnica) / outros métodos em vez do graph augmentation (válido).

RC

Notice the reference to the Algorithm construct

**Algorithm 5.1:** Quantum Multi-order Moments Embedding Protocol Strategy

**begin**

$nextBitrate \longleftarrow nextDownloadLevel$
$nextBitrate \longleftarrow GetNextBitrate()$
$cpuLoad \longleftarrow GetCpuLoad()$
$bitrateDelta \longleftarrow getBitrateDelta(currentBitrate, nextBitrate)$

**if** $bitrateDelta > maxThreshold$ **then**
  $SetBitrate(nextBitrate)$

**if** $minThreshold < bitrateDelta < maxThreshold$ **and** $numAttemps < 2$ **then**
  $numAttemps \longleftarrow numAttemps + 1$
**else if** $minThreshold < bitrateDelta < maxThreshold$ **and** $numAttemps = 2$ **then**
  $numAttemps \longleftarrow 0$
**else**
  $SetBitrate(nextBitrate)$
**if** $0 < bitrateDelta < minThreshold$ **and** $numAttemps < 3$ **then**
  $numAttemps \longleftarrow numAttemps + 1$
**else if** $0 < bitrateDelta < minThreshold$ **and** $numAttemps = 3$ **then**
  $SetBitrate(nextBitrate)$

**Multi-source streaming:** The app should support multi-source streaming media, i.e., “simultaneous” streaming of media, supported/complemented by QMME/Quantum Classification (QC) [52].

**Access Network independency:** The solution should provide the expected service over data networks such as QMME, QCP, etc.

## 5.2 Synthetic Dataset/Toy example

RC

Done: Gather the projective measurement statistics on a post-selected state. Had Python 3.13.1. Had to Install Python 3.11, since Qiskit's packages (qiskit 2.0.1) are not yet officially compatible with Python 3.13.

QRAM: perceber como funciona para as assumptions — Emmanuel Figures 5.1(a) and 5.1(b)

PM: $Qiskit$ uses little endian notation. Consider having a Summary of the Complete Algorithm/Protocol and Complexities for end-user applications as in https://www.nature.com/articles/s41534-024-00883-0.pdf

**(a)** Media Loading Window **(b)** Play-out Session UI

**Figure 5.1:** Complete User Interface

More Listing 5.1.

**Listing 5.1:** Example of a MPD file.

```
<?xml version="1.0" encoding="UTF-8"?>
<StreamInfo version="2.0">
    <Clip duration="PT01M0.00S">
        <BaseURL>videos/</BaseURL>
        <Description>svc_1</Description>
        <Representation mimeType="video/SVC" codecs="svc" frameRate="30.00" bandwidth="401.90"
            width="176" height="144" id="L0">
            <BaseURL>svc_1/</BaseURL>
            <SegmentInfo from="0" to="11" duration="PT5.00S">
                <BaseURL>svc_1-L0-</BaseURL>
            </SegmentInfo>
        </Representation>
        <Representation mimeType="video/SVC" codecs="svc" frameRate="30.00"
            bandwidth="1322.60"
            width="352" height="288" id="L1">
            <BaseURL>svc_1/</BaseURL>
            <SegmentInfo from="0" to="11" duration="PT5.00S">
                <BaseURL>svc_1-L1-</BaseURL>
            </SegmentInfo>
        </Representation>
    </Clip>
</StreamInfo>
```

RC

A figure with Subfigures

RC

A listing for XML code, with syntax highlighting

## 5.3 Real Network Data

# 6

# Concluding Remarks

## Contents


6.1 Conclusions . . . . . 47

6.2 Limitations and Future Work . . . . . 47

## 6.1 Conclusions

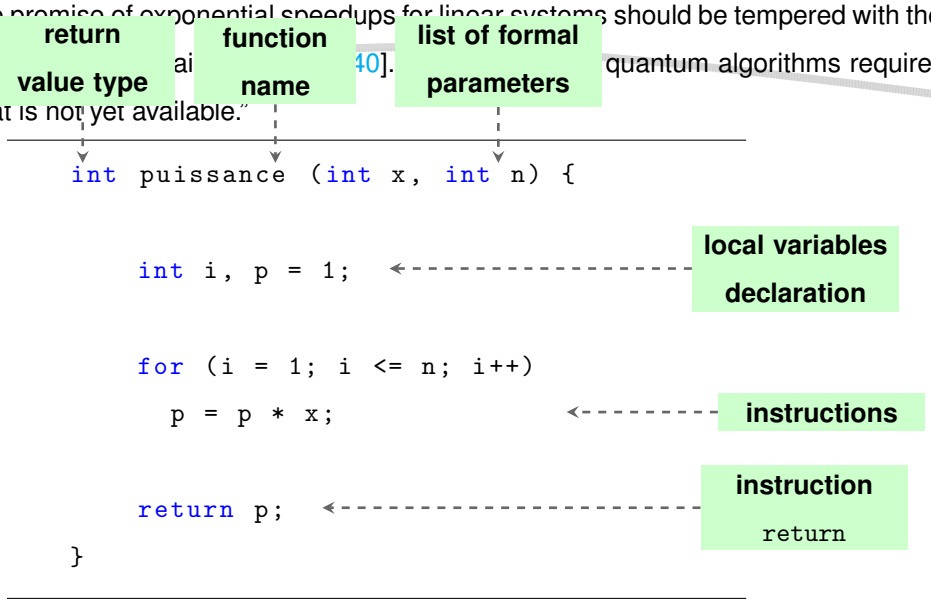
Quantum amplitude embeddings of neighbors' features moments can be used in other types of problems, as suggested in [4]. The reader might feel that the model used in this paper is in some respect artificial. In fact, this model is still far away from practically encountered situations. As was the one presented in [53].

PM
Tipo de problemas

## 6.2 Limitations and Future Work

There are several important caveats to the QMME and QC protocols. First, (...). Finally, although the algorithms scales as (...), current estimates of the cost of the algorithm for practical problems are prohibitive or not, which underlines the importance of investigating further improvements. "In general, the promise of exponential speedups for linear systems should be tempered with the realization that they ap ai 40]. quantum algorithms requires quantum hardware that is not yet available."

PM
I loved this term "caveat", used in [40]. Here, they pose 4 main challenges of quantum machine learning. Use them here or elsewhere!




**Listing 6.1:** A listing with a Tikz picture overlayed

And here another method (Listing 6.1) for mixing (overlay) a picture with a listing of code.

# Bibliography


[1] J. P. Moutinho, A. Melo, B. Coutinho, I. A. Kovács, and Y. Omar, "Quantum link prediction in complex networks," *Physical Review A*, vol. 107, no. 3, p. 032605, 2023.

[2] J. P. Moutinho, D. Magano, and B. Coutinho, "On the complexity of quantum link prediction in complex networks," *Scientific Reports*, vol. 14, no. 1, p. 1026, 2024.

[3] A. C. Gumpinger, K. Lage, H. Horn, and K. Borgwardt, "Prediction of cancer driver genes through network-based moment propagation of mutation scores," *Bioinformatics*, vol. 36, no. Supplement_1, pp. i508–i515, 2020.

[4] W. Bi, L. Du, Q. Fu, Y. Wang, S. Han, and D. Zhang, "Mm-gnn: Mix-moment graph neural network towards modeling neighborhood feature distribution," in *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*, 2023, pp. 132–140.

[5] D. K. Park, C. Blank, and F. Petruccione, "The theory of the quantum kernel-based binary classifier," *Physics Letters A*, vol. 384, no. 21, p. 126422, 2020.

[6] M. Soeken, M. Roetteler, N. Wiebe, and G. De Micheli, "Design automation and design space exploration for quantum computers," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*. Ieee, 2017, pp. 470–475.

[7] Apple, *HTTP Live Streaming Overview*, Apple Inc., 1 Infinite Loop, Cupertino, CA 95014, 408-996-1010 U.S., 2011. [Online]. Available: https://developer.apple.com/library/ios/documentation/networkinginternet/conceptual/streamingmediaguide/StreamingMediaGuide.pdf

[8] Adobe HTTP Dynamic Streaming. [Online]. Available: http://www.adobe.com/products/hds-dynamic-streaming.html

[9] Z. Alex. ISS Smooth Streaming Technical Overview. [Online]. Available: http://download.microsoft.com/download/4/2/4/4247C3AA-7105-4764-A8F9-321CB6C765EB/IIS_Smooth_Streaming_Technical_Overview.pdf

[10] Fraunhofer Heinrich-Hertz-Institute, “SVC: Scalable Extension of H.264/AVC,” 2013. [Online]. Available: http://www.hhi.fraunhofer.de/de/kompetenzfelder/image-processing/research-groups/image-video-coding/scalable-video-coding/svc-scalable-extension-of-h264avc.html

[11] ISO/IEC, “Information technology – Coding of audio-visual objects – Part 10: Advanced Video Coding,” International Organization for Standardization/International Electrotechnical Commission, International Standard ISO/IEC 14496-10:2012, Oct. 2012.

[12] P. Moscoso, “Interactive Internet TV Architecture Based on Scalable Video Coding,” Master’s thesis, Instituto Superior Técnico, May 2011.

[13] R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca, “Quantum algorithms revisited,” *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, vol. 454, no. 1969, pp. 339–354, 1998.

[14] D. Anguita, S. Ridella, F. Rivieccio, and R. Zunino, “Quantum optimization for training support vector machines,” *Neural Networks*, vol. 16, no. 5-6, pp. 763–770, 2003.

[15] S. Arora and B. Barak, *Computational complexity: a modern approach*. Cambridge University Press, 2009.

[16] J. M. Martyn, Z. M. Rossi, A. K. Tan, and I. L. Chuang, “Grand unification of quantum algorithms,” *PRX quantum*, vol. 2, no. 4, p. 040203, 2021.

[17] M. Möttönen, J. J. Vartiainen, V. Bergholm, and M. M. Salomaa, “Quantum circuits for general multiqubit gates,” *Physical review letters*, vol. 93, no. 13, p. 130502, 2004.

[18] B. MacAulay, A. Felts and Y. Fisher, “IP Streaming of MPEG-4 Native RTP vs MPEG-2 Transport Stream,” WHITEPAPER, October 2005. [Online]. Available: http://www.envivio.com/files/white-papers/RTPvsTS-v4.pdf

[19] H. Schwarz, D. Marpe, and T. Wiegand, “Overview of the Scalable Video Coding Extension of the H.264/AVC Standard,” *Circuits and Systems for Video Technology, IEEE Transactions on*, vol. 17, no. 9, pp. 1103–1120, 2007.

[20] R. A. Servedio and S. J. Gortler, “Equivalences and separations between quantum and classical learnability,” *SIAM Journal on Computing*, vol. 33, no. 5, pp. 1067–1092, 2004.

[21] J. Bankoski, J. Salonen, P. Wilins, and Y. Xu, “VP8 Data Format and Decoding Guide,” RFC 6386, IETF, RFC 6386, November 2011. [Online]. Available: http://tools.ietf.org/html/rfc6386

[22] Y.-H. Chiang, P. Huang, and H. Chen, “SVC or MDC? That’s the question,” in *Embedded Systems for Real-Time Multimedia (ESTIMedia), 2011 9th IEEE Symposium on*, 2011, pp. 76–82.

[23] D. P. DiVincenzo, “Two-bit gates are universal for quantum computation,” *Physical Review A*, vol. 51, no. 2, p. 1015, 1995.

[24] G. Verdon, T. McCourt, E. Luzhnica, V. Singh, S. Leichenauer, and J. Hidary, “Quantum graph neural networks,” *arXiv preprint arXiv:1909.12264*, 2019.

[25] M. Schuld and N. Killoran, “Quantum machine learning in feature hilbert spaces,” *Physical review letters*, vol. 122, no. 4, p. 040504, 2019, see Supplemental Material at https://link.aps.org/supplemental/10.1103/PhysRevLett.122.040504 for additional details.

[26] T. Hofmann, B. Schölkopf, and A. J. Smola, “Kernel methods in machine learning,” 2008.

[27] C. Blank, D. K. Park, J.-K. K. Rhee, and F. Petruccione, “Quantum classifier with tailored quantum kernel,” *npj Quantum Information*, vol. 6, no. 1, p. 41, 2020, see Supplemental Material at https://static-content.springer.com/esm/art%3A10.1038%2Fs41534-020-0272-6/MediaObjects/41534_2020_272_MOESM1_ESM.pdf for additional details.

[28] M. Schuld, A. Bocharov, K. M. Svore, and N. Wiebe, “Circuit-centric quantum classifiers,” *Physical Review A*, vol. 101, no. 3, p. 032308, 2020.

[29] J. R. McClean, J. Romero, R. Babbush, and A. Aspuru-Guzik, “The theory of variational hybrid quantum-classical algorithms,” *New Journal of Physics*, vol. 18, no. 2, p. 023023, 2016.

[30] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. AkashNarayanan, A. Asadi *et al.*, “Pennylane: Automatic differentiation of hybrid quantum-classical computations,” *arXiv preprint arXiv:1811.04968*, 2018.

[31] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, “Quantum circuit learning,” *Physical Review A*, vol. 98, no. 3, p. 032309, 2018.

[32] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, “Supervised learning with quantum-enhanced feature spaces,” *Nature*, vol. 567, no. 7747, pp. 209–212, 2019.

[33] M. J. Bremner, A. Montanaro, and D. J. Shepherd, “Average-case complexity versus approximate simulation of commuting quantum computations,” *Physical review letters*, vol. 117, no. 8, p. 080501, 2016.

[34] S. Lloyd, M. Mohseni, and P. Rebentrost, “Quantum algorithms for supervised and unsupervised machine learning,” *arXiv preprint arXiv:1307.0411*, 2013.

[35] P. Rebentrost, M. Mohseni, and S. Lloyd, “Quantum support vector machine for big data classification,” *Physical review letters*, vol. 113, no. 13, p. 130503, 2014.

[36] D. Camps, L. Lin, R. Van Beeumen, and C. Yang, "Explicit quantum circuits for block encodings of certain sparse matrices," *SIAM Journal on Matrix Analysis and Applications*, vol. 45, no. 1, pp. 801–827, 2024.

[37] L. Lin, "Lecture notes on quantum algorithms for scientific computation," *arXiv preprint arXiv:2201.08309*, 2022.

[38] Y. Tong, D. An, N. Wiebe, and L. Lin, "Fast inversion, preconditioned quantum linear system solvers, fast green's-function computation, and fast evaluation of matrix functions," *Physical Review A*, vol. 104, no. 3, p. 032422, 2021.

[39] A. Zaman, H. J. Morrell, and H. Y. Wong, "A step-by-step hhl algorithm walkthrough to enhance understanding of critical quantum computing concepts," *IEEE Access*, vol. 11, pp. 77 117–77 131, 2023.

[40] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.

[41] G. Sergioli, R. Giuntini, and H. Freytes, "A new quantum approach to binary classification," *PloS one*, vol. 14, no. 5, p. e0216224, 2019.

[42] M. Schuld, M. Fingerhuth, and F. Petruccione, "Implementing a distance-based classifier with a quantum interference circuit," *Europhysics Letters*, vol. 119, no. 6, p. 60002, 2017.

[43] D. C. Luna-Márquez, R. Pinto-Elías, T. Pérez-Becerra, A. Magadán-Salazar, N. González-Franco, and J. A. Fuentes-Pacheco, "knn classification using hausdorff metric with quantum-based euclidean distance," *Quantum Machine Intelligence*, vol. 7, no. 1, p. 54, 2025.

[44] L. Cincio, Y. Subaşı, A. T. Sornborger, and P. J. Coles, "Learning the quantum algorithm for state overlap," *New Journal of Physics*, vol. 20, no. 11, p. 113022, 2018.

[45] S. M. Pillay, I. Sinayskiy, E. Jembere, and F. Petruccione, "A multi-class quantum kernel-based classifier," *Advanced Quantum Technologies*, vol. 7, no. 1, p. 2300249, 2024.

[46] H. Buhrman, R. Cleve, J. Watrous, and R. De Wolf, "Quantum fingerprinting," *Physical review letters*, vol. 87, no. 16, p. 167902, 2001.

[47] C. Miquel, J. P. Paz, M. Saraceno, E. Knill, R. Laflamme, and C. Negrevergne, "Interpretation of tomography and spectroscopy as dual forms of quantum computation," *Nature*, vol. 418, no. 6893, pp. 59–62, 2002.

[48] Y. Wang and J. Liu, "A comprehensive review of quantum machine learning: from nisq to fault tolerance," *Reports on Progress in Physics*, 2024.

[49] C. T. Hann, C.-L. Zou, Y. Zhang, Y. Chu, R. J. Schoelkopf, S. M. Girvin, and L. Jiang, "Hardware-efficient quantum random access memory with hybrid quantum acoustic systems," *Physical review letters*, vol. 123, no. 25, p. 250501, 2019.

[50] M. S. Lawrence, P. Stojanov, C. H. Mermel, J. T. Robinson, L. A. Garraway, T. R. Golub, M. Meyerson, S. B. Gabriel, E. S. Lander, and G. Getz, "Discovery and saturation analysis of cancer genes across 21 tumour types," *Nature*, vol. 505, no. 7484, pp. 495–501, 2014.

[51] P. Erdös and P. Kelly, "The minimal regular graph containing a given graph," *American Mathematical Monthly*, vol. 70, no. 10, pp. 1074–1075, 1963.

[52] ISO/IEC, "Information technology – Dynamic adaptive streaming over HTTP (DASH) – Part 1: Media presentation description and segment formats," International Organization for Standardization/International Electrotechnical Commission, International Standard ISO/IEC FCD 23009-1:2012, Apr. 2012.

[53] M. Sasaki and A. Carlini, "Quantum learning and universal quantum matching machine," *Physical Review A*, vol. 66, no. 2, p. 022303, 2002.

[54] A. W. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for linear systems of equations," *Physical review letters*, vol. 103, no. 15, p. 150502, 2009.

# Code of Project

This inline MATLAB code `for i=1:3, disp('cool'); end;` uses the `\mcode{}` command.[1]

**Listing A.1:** `Matlab Function`

```
for i = 1:3
    if i >= 5 && a ~= b        % literate programming replacement
        disp('cool');          % comment with some LaTeXin it:  πx^2
    end
    [:,ind] = max(vec);
    x_last = x(1,end) - 1;
    v(end);
    ylabel('Voltage (μV)');
end
```

[1] MATLAB Works also in footnotes: `for i=1:3, disp('cool'); end;`

**Listing A.2:** `function.m`

```
%   Copyright 2010 The MathWorks, Inc.
function ObjTrack(position)
% #codegen
% First, setup the figure
numPts = 300;              % Process and plot 300 samples
figure;hold;grid;          % Prepare plot window
% Main loop
for idx = 1: numPts
    z = position(:,idx);   % Get the input data
    y = kalmanfilter(z);   % Call Kalman filter to estimate the position
    plot_trajectory(z,y);  % Plot the results
end
hold;
end   % of the function
```

**Listing A.3:** PYTHON Code

```
class TelgramRequestHandler(object):
    def handle(self):
        addr = self.client_address[0]          # Client IP-adress
        telgram = self.request.recv(1024)      # Recieve telgram
        print "From: %s, Received: %s" % (addr, telgram)
        return
```

# B

# Drafts and A Large Table

## B.1 Classification Unitary

In this representation, $\hat{y}_v^{\text{class}} = 1$ indicates node $v$ is classified as cancerous (i.e., a known cancer gene), and $\hat{y}_v^{\text{class}} = 0$ indicates a unknown-status node (likely healthy). Thus, the classifier learns to map input nodes to their corresponding health status based on the quantum measurement outcome.

Given $N$ training data points of the form $\{(\vec{x}_v, y_v) : \vec{x}_v \in \mathbb{R}^8, y_v \in \{0, 1\}\}_{v \in [N]}$, where $y_v$ indicates the class label of $\vec{x}_j$, the task reduces to finding the unitary $W$ as described in **??**. The goal is to map the input state $|\psi_{\text{in}}\rangle$, which encodes the amplitude-encoded training data, to the output state $|\psi_{\text{labels}}\rangle$, which uniformly encodes the known labels. This corresponds to solving the unitary **??**, a constrained linear mapping problem equation. Conceptually, this resembles solving a regression or classification problem via matrix inversion, but under the constraint that $W$ must be unitary.

To solve for $W$, we require labeled training data. The classification state $|\psi_{\text{QCP}}\rangle$ must be encoded from known labels. Once $W$ is determined, it can be applied to unseen nodes for classification.

*Directly calculating $W$ without training. Derive the eight-parameter $W$ analytically, e.g., using matrix inversion. More as a deterministic solution to a linear system than an iterative training model.*

*Kronecker product, not outer product!*

Solving **??** reduces to finding a unitary $W \in \mathbb{C}^{8\times 8}$, such that, for each fixed $v$, it maps the 8-dimensional feature vector $\vec{u}$ to one of the canonical base vectors, or not.

For more than 8 examples, I can't simultaneously do this unless all $\vec{u}_v$ lie in a subspace that allows to define such a mapping. A unitary exists if the set $\{\vec{u}\}$ is orthonormal (or can be orthogonalized), and each maps to a different label vector. A unitary can't map arbitrary feature vectors to arbitrary labels unless there's some structure (orthogonality, low rank, etc).

Imagine we are doing a linear separation of 2 points with 2 labels in a 2D-space. 2 points is exactly what we need to build a linear system of 2 equations with 2 incognitos $ax + b = 0$. The closed-form solution won't be scalable for large datasets (>8 nodes).

### B.1.1 HHL-inspired linear regression/classification (Amplitude-Encoding the Class)

The state generated by the QMMEP is given by

$$|\psi_{\text{QMME}}\rangle = \sum_{v=1}^{N}\sum_{j=1}^{8} u_{v,j}\,|v\rangle\,|j\rangle = \sum_{v=1}^{N} |v\rangle\,|\vec{u}_v\rangle\,, \tag{B.1}$$

which is the input for the QCP, with $|\vec{u}_v\rangle = \sum_{j=1}^{8} u_{v,j}\,|j\rangle$.

PM: Linear regression model (as in HHL). Then, threshold for linear classification (as in QSVMs for Big Data). For a general form for a hyperplane in the mapped space, see the nice paper [14], also about Quantum computing for SVM training.

The Linear System Problem (LSP) can be represented as $A\vec{x} = \vec{b}$ [54]. In our case,

$$A = (\vec{u}_1, \ldots, \vec{u}_v, \ldots, \vec{u}_N)^T, \tag{B.2}$$

with $A \in \mathbb{R}^{N\times 8}$, $\vec{b} \in \mathbb{C}^8$. Since our $A$ is not Hermitian, it can, in theory, be converted to a Hermitian matrix as $\tilde{A} = \begin{bmatrix} 0 & A \\ A^\dagger & 0 \end{bmatrix}$, with $A \in \mathbb{R}^{(N+8)\times(N+8)}$, $\vec{b} \in \mathbb{C}^(N+8)$.

The unitary $W$ shall act on the feature space and compute the corresponding label. In practice, $W$ may be implemented as a parameterized quantum circuit consisting of gates conditioned on the feature register. Specifically, for a given input vector $\vec{u}_v$, one may encode the label prediction into a dedicated classification qubit via a rotation gate, such as $R_y\left(\theta(\vec{u}_v)\right)|0\rangle = \cos\left(\frac{\theta(\vec{u}_v)}{2}\right)|0\rangle + \sin\left(\frac{\theta(\vec{u}_v)}{2}\right)|1\rangle = |\hat{y}_v^{\text{class}}\rangle$. To satisfy $W\,|\psi_{\text{QMME}}\rangle = |\psi_{\text{QCP}}\rangle$ as stated in **??**, consider $W = I^{\otimes N} \otimes I^{\otimes 8} \otimes R_y\left(\theta(\vec{u}_v)\right)$, where the final rotation acts on the classification qubit, conditioned (implicitly or explicitly) on the features. In practice, $R_y(\theta(\vec{u}_v))$ could be decomposed into a set of feature-controlled rotations (parameterized gates), depending on the encoding scheme. If the features were basis-encoded rather than amplitude-encoded, the classification logic could be implemented via controlled unitaries (e.g., controlled-$X$ or controlled

**Table B.1:** Example table

| Benchmark: ANN | **#Layers** (1) | **#Nets** (2) | #Nodes* $(3) = 8 \cdot (1) \cdot (2)$ | Critical path $(4) = 4 \cdot (1)$ | **Latency ($T_{iter}$)** (5) |
|---|---|---|---|---|---|
| A1 | **3–1501** | 1 | **24–12008** | **12–6004** | 4 |
| A2 | 501 | 1 | 4008 | 2004 | **2–2000** |
| A3 | 10 | **2–1024** | **160–81920** | 40 | 60† |
| A4 | 10 | 50 | 4000 | 40 | **80–1200** |
| **Benchmark: FFT** | **FFT size‡** (1) | *#Inputs* $(2) = 2^{(1)}$ | *#Nodes** $(3) = 10 \cdot (1) \cdot (2)$ | *Critical path* $(4) = 4 \cdot (1)$ | **Latency ($T_{iter}$)** (5) |
| F1 | **1–10** | 2–1024 | **20–102400** | 4–40 | 6–60† |
| F2 | **5** | 32 | 1600 | 20 | **40 – 1500** |
| **Benchmark: Random networks** | **#Types** (1) | **#Nodes** (2) | *#Networks* (3) | *Critical path* (4) | **Latency ($T_{iter}$)** (5) |
| R1 | 3 | 10–2000 | 500 | *variable* | (4) |
| R2 | 3 | 50 | 500 | *variable* | $(4) \times [1; \cdots; 20]$ |

* *Excluding constant nodes.*
† *Value kept proportional to the critical path:* $(5) = (4) * 1.5$.
‡ *A size of* $x$ *corresponds to a* $2^x$ *point FFT.*
*Values in bold indicate the parameter being varied.*

rotations) acting on the label qubit, with controls based on the feature basis states.

Matrix $A$ is encoded as the Hamiltonian of a unitary gate, and we expect the controlled-$U$ operation to be implemented by Hamiltonian simulation [39, 54]. However, in the latter, the matrix is derived for $U$ and then mapped to the Controlled$-U$ gate used in IBM-Q directly.

As Table B.1 shows, the data can be inserted from a file, in the case of a somehow complex structure. Notice the Table footnotes.

And now an example (Table B.2) of a table that extends to more than one page. Notice the repetition of the Caption (with indication that is continued) and of the Header, as well as the continuation text at the bottom.

**Table B.2:** Example of a very long table spreading in several pages

| Time (s) | Triple chosen | Other feasible triples |
|---|---|---|
| 0 | (1, 11, 13725) | (1, 12, 10980), (1, 13, 8235), (2, 2, 0), (3, 1, 0) |
| 2745 | (1, 12, 10980) | (1, 13, 8235), (2, 2, 0), (2, 3, 0), (3, 1, 0) |
| 5490 | (1, 12, 13725) | (2, 2, 2745), (2, 3, 0), (3, 1, 0) |
| 8235 | (1, 12, 16470) | (1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0) |

An example of a large Table that autofits the size to the page margins is illustrated in Table B.3. Please notice the text size that is shrunken in order fot the table to adjust to the page:

**Table B.3:** Sample Table.

| URL | First Time Visit | Last Time Visit | URL Counts | Value | Reference |
|---|---|---|---|---|---|
| https://web.facebook.com/ | 1521241972 | 1522351859 | 177 | 56640 | [facebook-2021] |